Sharpness Lean Formalization Inventory

Definitions, theorem statements, and dependency graph

Generated from the local Lean sources on 2026-06-25 00:42.

Proof Completion Check

- `lake env lean Sharpness/Main.lean`: passed.
- `lake build`: passed. The emitted messages were style/header linter warnings, not proof errors.
- Source scan over Lean files found no `sorry`, `axiom`, `constant`, `unsafe`, `admit`, or global `set_option autoImplicit true`.
- `#print axioms Sharpness.sharpness_zd` returned exactly {propext, Classical.choice, Quot.sound}. In particular, no `sorryAx` appears.

The inventory below covers 19 Sharpness modules, 85 definitions/structures/abbreviations, and 283 theorems/lemmas, including 166 private supporting declarations.

Final Theorem as a Mathematical Statement

For every dimension $d \geq 2$, the Lean theorem `Sharpness.sharpness_zd` proves the conjunction of the following two statements.

$$\forall p, \quad 0 \leq p < p_{\mathrm{c}}(d) \implies \exists c > 0,\ \forall n \in \mathbb{N},\ \mathbb{P}_p(0 \leftrightarrow \partial\Lambda_n) \leq \exp(-cn),$$

$$\forall p, \quad p_{\mathrm{c}}(d) < p < 1 \implies \theta_d(p) \geq \frac{p - p_{\mathrm{c}}(d)}{p(1 - p_{\mathrm{c}}(d))}.$$

Here $\mathbb{P}_p(0 \leftrightarrow \partial\Lambda_n)$ is formalized as `boxExitProb d p n`; $\theta_d(p)$ is formalized as `theta d p`; and $p_{\mathrm{c}}(d)$ is formalized as `pCrit d`.

Core Public Theorems

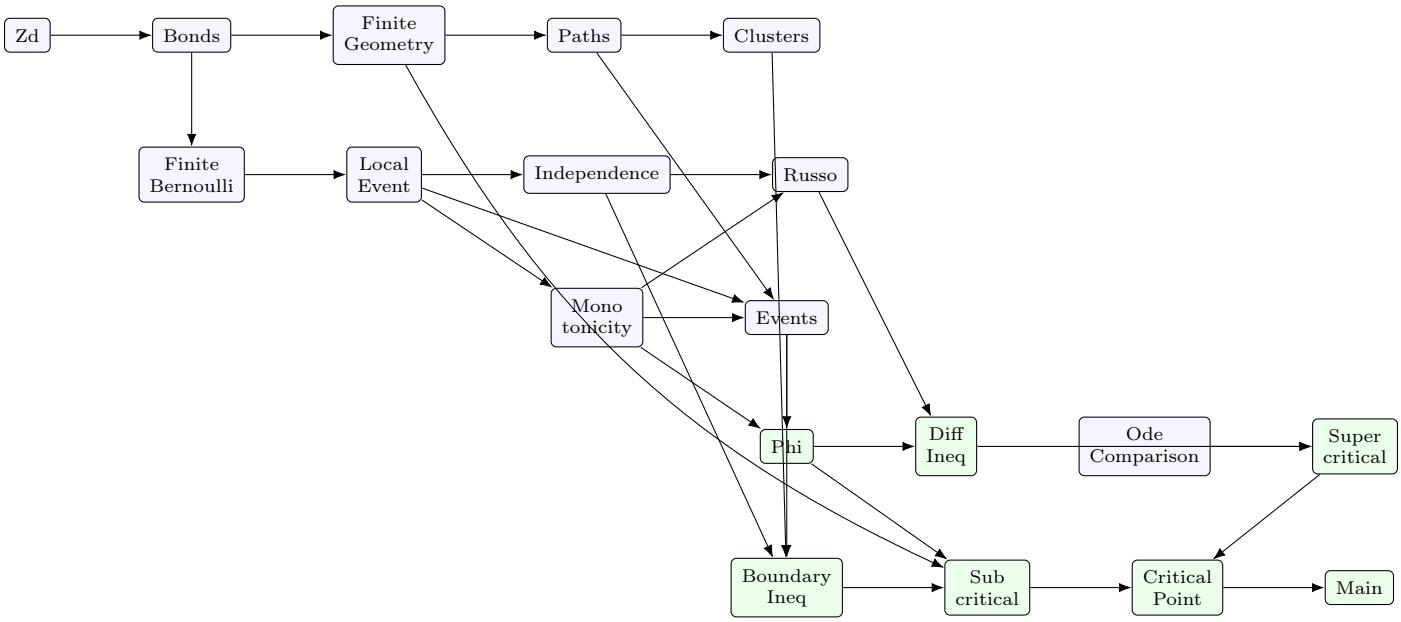
Lean theorem	Mathematical statement
`russo_closed_pivotal`	For an increasing finite local event E, $0 < p < 1$, the derivative of $q \mapsto \mathbb{P}_q(E)$ at p is $$\frac{1}{1-p} \sum_{e \in \mathrm{supp}(E)} \mathbb{P}_p(e \text{ is closed-pivotal for } E).$$
`differential_inequality`	For a finite set $\Lambda \ni 0$ and $0 < p < 1$, $$\frac{d}{dp}\mathbb{P}_p(0 \leftrightarrow \partial\Lambda) \geq \frac{\phi_{\min}(p, \Lambda)}{p(1-p)}\big(1 - \mathbb{P}_p(0 \leftrightarrow \partial\Lambda)\big).$$
`ode_log_comparison`	If $f < 1$ on $[q, p]$, $0 < q < p < 1$, and $$f'(t) \geq \frac{1 - f(t)}{t(1-t)} \quad (q < t < p),$$ then $$1 - f(p) \leq (1 - f(q))\frac{q(1-p)}{p(1-q)}.$$

Lean theorem	Mathematical statement
`supercritical_lower_bound_above_pTilde`	If $\widetilde{p}(d) < p < 1$, then $$\theta_d(p) \geq \frac{p - \widetilde{p}(d)}{p(1 - \widetilde{p}(d))}.$$
`boundary_inequality`	For finite $B, S \subseteq T$, $u \in S$, and $B \cap S = \varnothing$, $$\mathbb{P}_p(u \leftrightarrow B \text{ in } T) \leq \sum_{(x,y)\in\partial S} p\,\mathbb{P}_p(u \leftrightarrow x \text{ in } S)\,\mathbb{P}_p(y \leftrightarrow B \text{ in } T).$$
`exponential_decay_below_pTilde`	If $0 \leq p < \widetilde{p}(d)$, then there is $c > 0$ such that $$\mathbb{P}_p(0 \leftrightarrow \partial\Lambda_n) \leq e^{-cn} \quad \text{for all } n.$$
`pTilde_eq_pCrit`	For $d > 0$, the finite-set critical point equals the θ-critical point: $$\widetilde{p}(d) = p_c(d).$$
`exponential_decay_below_pCrit`	If $d > 0$ and $0 \leq p < p_c(d)$, then finite box exit probabilities decay exponentially.
`supercritical_lower_bound_above_pCrit`	If $d > 0$ and $p_c(d) < p < 1$, then $$\theta_d(p) \geq \frac{p - p_c(d)}{p(1 - p_c(d))}.$$
`sharpness_zd`	Combines the two $p_c(d)$-form conclusions above for all $d \geq 2$.

Important Definitions

Lean definition	Mathematical meaning
`Vertex d`	The lattice $\mathbb{Z}^d$, represented as functions Fin $d \to \mathbb{Z}$.
`Bond d`	An unoriented nearest-neighbor edge, represented by its two endpoint carrier.
`Config d`	A bond configuration $\omega : E(\mathbb{Z}^d) \to \{\text{false}, \text{true}\}$.
`bernProb p A`	The finite Bernoulli product probability of a Boolean event A.
`LocalEvent d`	A configuration event with an explicit finite bond support.
`Prob p E`	The probability of a local event E, computed on its finite support.
`ExitPred Lam omega`	The event that 0 connects inside Λ to an open boundary edge leaving Λ.
`exitProb p Lam h0`	The finite-volume exit probability $\mathbb{P}_p(0 \leftrightarrow \partial\Lambda)$.
`boxExitProb d p n`	The exit probability from the l^1-ball Λ_n.
`theta d p`	The infimum over all finite box exit probabilities, i.e. the decreasing-limit proxy.
`phi p S`	The finite-set criterion quantity $p \sum_{(x,y)\in\partial S} \mathbb{P}_p(0 \leftrightarrow x \text{ in } S)$.
`pTilde d`	The supremum of parameters satisfying the finite-set criterion.
`pCrit d`	The infimum of parameters in $[0, 1]$ for which $\theta_d(p) > 0$.

Module Dependency Graph



Actual Import Edges

- `Sharpness.Zd` → `Sharpness.Bonds`
- `Sharpness.Bonds` → `Sharpness.FiniteGeometry`
- `Sharpness.FiniteGeometry` → `Sharpness.Paths`
- `Sharpness.LocalEvent` → `Sharpness.Clusters`
- `Sharpness.Paths` → `Sharpness.Clusters`
- `Sharpness.Bonds` → `Sharpness.FiniteBernoulli`
- `Sharpness.FiniteBernoulli` → `Sharpness.LocalEvent`
- `Sharpness.LocalEvent` → `Sharpness.Independence`
- `Sharpness.LocalEvent` → `Sharpness.Monotonicity`
- `Sharpness.Independence` → `Sharpness.Russo`
- `Sharpness.Monotonicity` → `Sharpness.Russo`
- `Sharpness.LocalEvent` → `Sharpness.Events`
- `Sharpness.Monotonicity` → `Sharpness.Events`
- `Sharpness.Paths` → `Sharpness.Events`
- `Sharpness.Events` → `Sharpness.Phi`
- `Sharpness.Monotonicity` → `Sharpness.Phi`
- `Sharpness.Phi` → `Sharpness.DiffIneq`
- `Sharpness.Russo` → `Sharpness.DiffIneq`
- `Sharpness.DiffIneq` → `Sharpness.Supercritical`
- `Sharpness.OdeComparison` → `Sharpness.Supercritical`
- `Sharpness.Clusters` → `Sharpness.BoundaryIneq`
- `Sharpness.Events` → `Sharpness.BoundaryIneq`
- `Sharpness.Independence` → `Sharpness.BoundaryIneq`
- `Sharpness.BoundaryIneq` → `Sharpness.Subcritical`
- `Sharpness.FiniteGeometry` → `Sharpness.Subcritical`
- `Sharpness.Phi` → `Sharpness.Subcritical`
- `Sharpness.Subcritical` → `Sharpness.CriticalPoint`
- `Sharpness.Supercritical` → `Sharpness.CriticalPoint`
- `Sharpness.CriticalPoint` → `Sharpness.Main`

Per-File Declaration Inventory

`Sharpness/Zd.lean`

Imports. `Mathlib`.

Definitions, structures, and abbreviations.

Name	Statement or signature
Vertex	line 8: abbrev Vertex (d : Nat) -- Vertices of nearest-neighbor bond percolation on ‘Z^d‘.
l1	line 11: def l1 {d : Nat} (x : Vertex d) : Nat -- The ‘l^1‘ norm on ‘Z^d‘, valued in ‘Nat‘.
Adj	line 15: def Adj {d : Nat} (x y : Vertex d) : Prop -- Nearest-neighbor adjacency on ‘Z^d‘.
translateVertex	line 19: def translateVertex {d : Nat} (a x : Vertex d) : Vertex d -- Translate a vertex by another vertex.
coordStep	line 23: def coordStep {d : Nat} (i : Fin d) (positive : Bool) : Vertex d -- A single positive or negative coordinate step.
coordRange [noncomputable]	line 27: noncomputable def coordRange (n : Nat) : Finset Int -- Integer coordinates available in the cube containing the ‘l^1‘ ball.
cube [noncomputable]	line 31: noncomputable def cube (d n : Nat) : Finset (Vertex d) -- The finite coordinate cube ‘[-n,n]^d‘.
ball [noncomputable]	line 41: noncomputable def ball (d n : Nat) : Finset (Vertex d) -- The finite ‘l^1‘ box ‘Lambda_n = {x : Z^d | |x||_1 <= n}‘.
neighbors [noncomputable]	line 45: noncomputable def neighbors {d : Nat} (x : Vertex d) : Finset (Vertex d) -- The finite set of nearest-neighbor candidates of a vertex.

Theorems and lemmas.

Name	Statement or signature
translateVertex_apply	line 49: theorem translateVertex_apply {d : Nat} (a x : Vertex d) (i : Fin d) : translateVertex a x i = x i + a i
l1_zero	line 53: theorem l1_zero {d : Nat} : l1 (0 : Vertex d) = 0
l1_neg	line 56: theorem l1_neg {d : Nat} (x : Vertex d) : l1 (-x) = l1 x
l1_add_le	line 59: theorem l1_add_le {d : Nat} (x y : Vertex d) : l1 (x + y) <= l1 x + l1 y
l1_sub_comm	line 72: theorem l1_sub_comm {d : Nat} (x y : Vertex d) : l1 (x - y) = l1 (y - x)
adj_symm	line 79: theorem adj_symm {d : Nat} {x y : Vertex d} (hxy : Adj x y) : Adj y x
ne_of_adj	line 82: theorem ne_of_adj {d : Nat} {x y : Vertex d} (hxy : Adj x y) : x != y
mem_cube_iff	line 87: theorem mem_cube_iff {d n : Nat} {x : Vertex d} : x in cube d n <-> forall i : Fin d, x i in coordRange n
coord_natAbs_le_l1	line 103: theorem coord_natAbs_le_l1 {d : Nat} (x : Vertex d) (i : Fin d) : Int.natAbs (x i) <= l1 x
coord_mem_coordRange_of_l1_le	line 111: theorem coord_mem_coordRange_of_l1_le {d n : Nat} {x : Vertex d} (hx : l1 x <= n) (i : Fin d) : x i in coordRange n
mem_ball_iff	line 123: theorem mem_ball_iff {d n : Nat} {x : Vertex d} : x in ball d n <-> l1 x <= n -- Characterize membership in the finite ‘l^1‘ ball constructed from the bounded cube.
zero_mem_ball	line 134: theorem zero_mem_ball (d n : Nat) : (0 : Vertex d) in ball d n -- The origin belongs to every finite ‘l^1‘ ball.
adj_translate_iff	line 139: theorem adj_translate_iff {d : Nat} {a x y : Vertex d} : Adj (translateVertex a x) (translateVertex a y) <-> Adj x y -- Nearest-neighbor adjacency is invariant under translations.
mem_neighbors_iff	line 148: theorem mem_neighbors_iff {d : Nat} {x y : Vertex d} : y in neighbors x <-> Adj x y -- Nearest-neighbor candidates are exactly adjacent vertices.
translate_ball_exit	line 170: theorem translate_ball_exit {d n L : Nat} {y z : Vertex d} (hy : y in ball d L) (hz : z notin ball d n) (hLn : L <= n) : z - y notin ball d (n - L) -- The triangle estimate used to compare translated exit events.

Sharpness/Bonds.lean

Imports. Sharpness.Zd.

Definitions, structures, and abbreviations.

Name	Statement or signature
Bond	line 6: structure Bond (d : Nat) -- An unoriented nearest-neighbor bond, represented by its two-point carrier.
Config	line 13: abbrev Config (d : Nat) -- Global percolation configurations, indexed by unoriented bonds.
bondOfAdj	line 33: def bondOfAdj {d : Nat} {x y : Vertex d} (hxy : Adj x y) : Bond d -- The unoriented bond associated to an adjacent ordered pair.
OrientedEdge	line 39: abbrev OrientedEdge (d : Nat) -- An oriented edge is a pair of vertices. Boundary sums use this orientation.
orientedBoundary [noncomputable]	line 42: noncomputable def orientedBoundary {d : Nat} (S : Finset (Vertex d)) : Finset (OrientedEdge d) -- Oriented boundary edges from a finite vertex set to its complement.
OEdgeBoundary	line 80: structure OEdgeBoundary {d : Nat} (S : Finset (Vertex d)) -- A boundary edge with its orientation and membership proofs.
OEdgeBoundary.bond	line 88: def OEdgeBoundary.bond {d : Nat} {S : Finset (Vertex d)} (e : OEdgeBoundary S) : Bond d -- Forget the orientation of a boundary edge.
internalBonds [noncomputable]	line 100: noncomputable def internalBonds {d : Nat} (S : Finset (Vertex d)) : Finset (Bond d) -- Bonds with both endpoints in a finite vertex set.
exitBonds [noncomputable]	line 121: noncomputable def exitBonds {d : Nat} (S : Finset (Vertex d)) : Finset (Bond d) -- Unoriented bonds crossing the boundary of a finite vertex set.

Theorems and lemmas.

Name	Statement or signature
bondOfAdj_card_two	line 16: theorem bondOfAdj_card_two {d : Nat} {x y : Vertex d} (hxy : Adj x y) : ({x, y} : Finset (Vertex d)).card = 2 -- Adjacent vertices form a two-point finite carrier.
bondOfAdj_adj_pair	line 21: theorem bondOfAdj_adj_pair {d : Nat} {x y : Vertex d} (hxy : Adj x y) : forall u, u in ({x, y} : Finset (Vertex d)) -> forall v, v in ({x, y} : Finset (Vertex d)) -> u != v -> Adj u v -- Every distinct pair in the carrier ‘{x,y}‘ is adjacent when ‘x‘ and ‘y‘ are adjacent.
mem_orientedBoundary_iff	line 52: theorem mem_orientedBoundary_iff {d : Nat} {S : Finset (Vertex d)} {e : OrientedEdge d} : e in orientedBoundary S <-> e.1 in S /\ e.2 notin S /\ Adj e.1 e.2 -- The finite neighbor construction exactly enumerates oriented nearest-neighbor boundary edges.
orientedBoundary_adj	line 75: theorem orientedBoundary_adj {d : Nat} {S : Finset (Vertex d)} {e : OrientedEdge d} (he : e in orientedBoundary S) : Adj e.1 e.2 -- Adjacency proof extracted from membership in the oriented boundary.
bondOfAdj_same	line 91: theorem bondOfAdj_same {d : Nat} {x y : Vertex d} {h h' : Adj x y} : bondOfAdj h = bondOfAdj h'
bondOfAdj_symm	line 95: theorem bondOfAdj_symm {d : Nat} {x y : Vertex d} (hxy : Adj x y) : bondOfAdj (adj_symm hxy) = bondOfAdj hxy
bondOfAdj_mem_internalBonds	line 105: theorem bondOfAdj_mem_internalBonds {d : Nat} {S : Finset (Vertex d)} {x y : Vertex d} (hx : x in S) (hy : y in S) (hxy : Adj x y) : bondOfAdj hxy in internalBonds S
bondOfAdj_mem_exitBonds	line 125: theorem bondOfAdj_mem_exitBonds {d : Nat} {S : Finset (Vertex d)} {e : OrientedEdge d} (he : e in orientedBoundary S) : bondOfAdj (orientedBoundary_adj he) in exitBonds S
disjoint_internal_exit	line 134: theorem disjoint_internal_exit {d : Nat} (S : Finset (Vertex d)) : Disjoint (internalBonds S) (exitBonds S) -- Internal and exit bonds of the same finite set are disjoint.

Sharpness/FiniteGeometry.lean

Imports. Sharpness.Bonds.

Definitions, structures, and abbreviations.

Name	Statement or signature
boxInternalBonds [noncomputable]	line 6: noncomputable def boxInternalBonds (d n : Nat) : Finset (Bond d) -- Internal bonds of the box ‘Lambda_n‘.

Name	Statement or signature
boxExitBonds [noncomputable]	line 10: noncomputable def boxExitBonds (d n : Nat) : Finset (Bond d) -- Exit bonds crossing from ‘Lambda_n‘ to its complement.

Theorems and lemmas.

Name	Statement or signature
boundary_endpoint_mem_ball	line 18: theorem boundary_endpoint_mem_ball {d L : Nat} {S : Finset (Vertex d)} {e : OrientedEdge d} (hL : 0 < L) (hS : forall x, x in S -> x in ball d (L - 1)) (he : e in orientedBoundary S) : e.2 in ball d L -- Boundary endpoints of a finite set contained in ‘Lambda_(L-1)‘ lie in ‘Lambda_L‘. The positivity hypothesis is needed because ‘Nat‘ subtraction saturates: at ‘L = 0‘, ‘L - 1 = 0‘, while a boundary endpoint of ‘{0}‘ need not be in ‘Lambda_0‘.

Sharpness/Paths.lean

Imports. Sharpness.FiniteGeometry.

Definitions, structures, and abbreviations.

Name	Statement or signature
IsPath	line 6: def IsPath {d : Nat} (gamma : List (Vertex d)) : Prop -- A list of vertices with adjacent consecutive entries.
PathFromTo	line 10: def PathFromTo {d : Nat} (gamma : List (Vertex d)) (x y : Vertex d) : Prop -- A path from ‘x‘ to ‘y‘, represented as a finite list.
PathIn	line 14: def PathIn {d : Nat} (S : Finset (Vertex d)) (gamma : List (Vertex d)) : Prop -- Every vertex of the path lies in the finite set ‘S‘.
OpenPath	line 18: def OpenPath {d : Nat} (omega : Config d) (gamma : List (Vertex d)) : Prop -- Every consecutive bond of the path is open in the configuration.
ConnIn	line 22: def ConnIn {d : Nat} (omega : Config d) (S : Finset (Vertex d)) (x y : Vertex d) : Prop -- Open connectivity inside a finite vertex set.

Theorems and lemmas.

Name	Statement or signature
first_exit_step_of_path	line 28: theorem first_exit_step_of_path {d : Nat} {S : Finset (Vertex d)} {gamma : List (Vertex d)} {x y : Vertex d} (hgamma : PathFromTo gamma x y) (hx : x in S) (hy : y notin S) : exists u v : Vertex d, u in gamma /\ v in gamma /\ Adj u v /\ u in S /\ v notin S -- A path that starts in ‘S‘ and ends outside ‘S‘ has a crossing step.
last_exit_step_of_path	line 62: theorem last_exit_step_of_path {d : Nat} {S : Finset (Vertex d)} {gamma : List (Vertex d)} {x y : Vertex d} (hgamma : PathFromTo gamma x y) (hx : x in S) (hy : y notin S) : exists u v : Vertex d, u in gamma /\ v in gamma /\ Adj u v /\ u in S /\ v notin S -- A path crossing from ‘S‘ to its complement has a boundary step; this alias is used for last-exit arguments before the later proof refines the chosen crossing.
isPath_reverse	line 69: theorem isPath_reverse {d : Nat} {gamma : List (Vertex d)} (hgamma : IsPath gamma) : IsPath gamma.reverse
pathFromTo_reverse	line 74: theorem pathFromTo_reverse {d : Nat} {gamma : List (Vertex d)} {x y : Vertex d} (hgamma : PathFromTo gamma x y) : PathFromTo gamma.reverse y x
pathIn_reverse	line 82: theorem pathIn_reverse {d : Nat} {S : Finset (Vertex d)} {gamma : List (Vertex d)} (hgamma : PathIn S gamma) : PathIn S gamma.reverse
openPath_reverse	line 87: theorem openPath_reverse {d : Nat} {omega : Config d} {gamma : List (Vertex d)} (hgamma : OpenPath omega gamma) : OpenPath omega gamma.reverse
connIn_symm	line 95: theorem connIn_symm {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {x y : Vertex d} (hxy : ConnIn omega S x y) : ConnIn omega S y x -- Reversing a path preserves open connectivity.
isPath_translate	line 101: theorem isPath_translate {d : Nat} {a : Vertex d} {gamma : List (Vertex d)} (hgamma : IsPath gamma) : IsPath (gamma.map (translateVertex a))

Name	Statement or signature
pathFromTo_translate	line 106: theorem pathFromTo_translate {d : Nat} {a x y : Vertex d} {gamma : List (Vertex d)} (hgamma : PathFromTo gamma x y) : PathFromTo (gamma.map (translateVertex a)) (translateVertex a x) (translateVertex a y)
pathIn_translate	line 113: theorem pathIn_translate {d : Nat} {a : Vertex d} {S : Finset (Vertex d)} {gamma : List (Vertex d)} (hgamma : PathIn S gamma) : PathIn (S.image (translateVertex a)) (gamma.map (translateVertex a))
openPath_translate_of_ edge_open	line 121: theorem openPath_translate_of_edge_open {d : Nat} {omega : Config d} {a : Vertex d} {gamma : List (Vertex d)} (hopen_translate : forall {u v : Vertex d} (huv : Adj u v), omega (bondOfAdj huv) = true -> omega (bondOfAdj ((adj_translate_iff (a
connIn_translate	line 134: theorem connIn_translate {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {a x y : Vertex d} (hopen_translate : forall {u v : Vertex d} (huv : Adj u v), omega (bondOfAdj huv) = true -> omega (bondOfAdj ((adj_translate_iff (a -- Translated paths preserve open connectivity when translated open edges remain open.

Sharpness/Clusters.lean

Imports. Sharpness.LocalEvent, Sharpness.Paths.

Definitions, structures, and abbreviations.

Name	Statement or signature
clusterIn [noncomputable]	line 7: noncomputable def clusterIn {d : Nat} (omega : Config d) (S : Finset (Vertex d)) (u : Vertex d) : Finset (Vertex d) -- The open cluster of ‘u‘ inside a finite vertex set ‘S‘.
ClusterEqPred	line 13: def ClusterEqPred {d : Nat} (S C : Finset (Vertex d)) (u : Vertex d) (omega : Config d) : Prop -- Predicate that the finite cluster inside ‘S‘ is exactly ‘C‘.
clusterIncidentBonds [noncomputable]	line 18: noncomputable def clusterIncidentBonds {d : Nat} (S C : Finset (Vertex d)) : Finset (Bond d) -- Bonds inside ‘S‘ with at least one endpoint in the candidate cluster ‘C‘.

Theorems and lemmas.

Name	Statement or signature
bondOfAdj_mem_ clusterIncidentBonds [private]	line 25: private theorem bondOfAdj_mem_clusterIncidentBonds {d : Nat} {S C : Finset (Vertex d)} {x y : Vertex d} (hxS : x in S) (hyS : y in S) (hxy : Adj x y) (hinc : x in C \/ y in C) : bondOfAdj hxy in clusterIncidentBonds S C
connIn_single [private]	line 42: private theorem connIn_single {d : Nat} {omega : Config d} {S : Finset (Vertex d) } {x : Vertex d} (hx : x in S) : ConnIn omega S x x
connIn_append_open_adj_ same [private]	line 51: private theorem connIn_append_open_adj_same {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {x y z : Vertex d} (hzS : z in S) (hyz : Adj y z) (hopen : omega (bondOfAdj hyz) = true) (hconn : ConnIn omega S x y) : ConnIn omega S x z
mem_cluster_of_conn [private]	line 88: private theorem mem_cluster_of_conn {d : Nat} {omega : Config d} {S C : Finset (Vertex d)} {u x : Vertex d} (hC : clusterIn omega S u = C) (hconn : ConnIn omega S u x) : x in C
conn_of_mem_cluster [private]	line 99: private theorem conn_of_mem_cluster {d : Nat} {omega : Config d} {S C : Finset (Vertex d)} {u x : Vertex d} (hC : clusterIn omega S u = C) (hxC : x in C) : ConnIn omega S u x
target_mem_of_connIn [private]	line 107: private theorem target_mem_of_connIn {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {u x : Vertex d} (hconn : ConnIn omega S u x) : x in S
mem_clusterIn_of_conn [private]	line 113: private theorem mem_clusterIn_of_conn {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {u x : Vertex d} (hconn : ConnIn omega S u x) : x in clusterIn omega S u
openPath_transfer_from_ cluster_eq [private]	line 119: private theorem openPath_transfer_from_cluster_eq {d : Nat} {omega omega’ : Config d} {S C : Finset (Vertex d)} {u : Vertex d} (hsame : forall e, e in clusterIncidentBonds S C -> omega e = omega’ e) (hC : clusterIn omega S u = C) {gamma : List (Vertex d)} (hS : PathIn S gamma) (hchain : IsPath gamma) (hopen : OpenPath omega gamma) (hheadC : forall a, gamma.head? = some a -> a in C) : OpenPath omega’ gamma

Name	Statement or signature
openPath_transfer_to_cluster_eq [private]	line 161: private theorem openPath_transfer_to_cluster_eq {d : Nat} {omega omega' : Config d} {S C : Finset (Vertex d)} {u : Vertex d} (hsame : forall e, e in clusterIncidentBonds S C -> omega e = omega' e) (hC : clusterIn omega S u = C) {gamma : List (Vertex d)} (hS : PathIn S gamma) (hchain : IsPath gamma) (hopen' : OpenPath omega' gamma) (hheadC : forall a, gamma.head? = some a -> a in C) : OpenPath omega gamma
connIn_transfer_of_cluster_eq [private]	line 204: private theorem connIn_transfer_of_cluster_eq {d : Nat} {omega omega' : Config d} {S C : Finset (Vertex d)} {u x : Vertex d} (hsame : forall e, e in clusterIncidentBonds S C -> omega e = omega' e) (hC : clusterIn omega S u = C) (hconn : ConnIn omega S u x) : ConnIn omega' S u x
connIn_other_subset_cluster [private]	line 223: private theorem connIn_other_subset_cluster {d : Nat} {omega omega' : Config d} {S C : Finset (Vertex d)} {u x : Vertex d} (hsame : forall e, e in clusterIncidentBonds S C -> omega e = omega' e) (hC : clusterIn omega S u = C) (hconn' : ConnIn omega' S u x) : x in C
cluster_eq_dependsOn_incident	line 244: theorem cluster_eq_dependsOn_incident {d : Nat} (S C : Finset (Vertex d)) (u : Vertex d) : DependsOn (clusterIncidentBonds S C) (ClusterEqPred S C u)

Sharpness/FiniteBernoulli.lean

Imports. Sharpness.Bonds.

Definitions, structures, and abbreviations.

Name	Statement or signature
bernWeight [noncomputable]	line 12: noncomputable def bernWeight (p : Real) (sigma : alpha -> Bool) : Real -- Bernoulli product weight on an arbitrary finite coordinate type.
bernProb [noncomputable]	line 16: noncomputable def bernProb (p : Real) (A : (alpha -> Bool) -> Prop) : Real -- Bernoulli product probability on an arbitrary finite coordinate type.
weight [noncomputable]	line 279: noncomputable def weight {d : Nat} (p : Real) (F : Finset (Bond d)) (sigma : F -> Bool) : Real -- Bernoulli product weight of an assignment on a finite bond support.
ProbOn [noncomputable]	line 284: noncomputable def ProbOn {d : Nat} (p : Real) (F : Finset (Bond d)) (A : (F -> Bool) -> Prop) : Real -- Finite Bernoulli product probability on an explicit finite bond support.

Theorems and lemmas.

Name	Statement or signature
bernWeight_nonneg	line 22: theorem bernWeight_nonneg {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (sigma : alpha -> Bool) : 0 <= bernWeight p sigma
bernWeight_total	line 33: theorem bernWeight_total (p : Real) : (Finset.univ.sum fun sigma : alpha -> Bool => bernWeight p sigma) = 1
bernProb_nonneg	line 49: theorem bernProb_nonneg {p : Real} {A : (alpha -> Bool) -> Prop} (hp0 : 0 <= p) (hp1 : p <= 1) : 0 <= bernProb p A
bernProb_le_one	line 59: theorem bernProb_le_one {p : Real} {A : (alpha -> Bool) -> Prop} (hp0 : 0 <= p) (hp1 : p <= 1) : bernProb p A <= 1
bernProb_univ	line 70: theorem bernProb_univ {p : Real} : bernProb p (fun _ : alpha -> Bool => True) = 1
bernProb_compl	line 75: theorem bernProb_compl {p : Real} {A : (alpha -> Bool) -> Prop} : bernProb p (fun sigma => not A sigma) = 1 - bernProb p A
bernProb_mono	line 85: theorem bernProb_mono {p : Real} {A B : (alpha -> Bool) -> Prop} (hAB : forall sigma, A sigma -> B sigma) (hp0 : 0 <= p) (hp1 : p <= 1) : bernProb p A <= bernProb p B
bernProb_union_bound	line 99: theorem bernProb_union_bound {p : Real} {A B : (alpha -> Bool) -> Prop} (hp0 : 0 <= p) (hp1 : p <= 1) : bernProb p (fun sigma => A sigma \/ B sigma) <= bernProb p A + bernProb p B
bernWeight_reindex	line 110: theorem bernWeight_reindex {beta : Type*} [DecidableEq beta] [Fintype beta] (p : Real) (e : alpha ~= beta) (sigma : beta -> Bool) : bernWeight p (fun a : alpha => sigma (e a)) = bernWeight p sigma

Name	Statement or signature
bernProb_reindex	line 120: theorem bernProb_reindex {beta : Type*} [DecidableEq beta] [Fintype beta] (p : Real) (e : alpha ~= beta) (A : (beta -> Bool) -> Prop) : bernProb p (fun sigma : alpha -> Bool => A (fun b : beta => sigma (e.symm b))) = bernProb p A
bernWeight_sum	line 149: theorem bernWeight_sum {beta : Type*} [DecidableEq beta] [Fintype beta] (p : Real) (sigma : alpha + beta -> Bool) : bernWeight p sigma = bernWeight p (fun a : alpha => sigma (Sum.inl a)) * bernWeight p (fun b : beta => sigma (Sum.inr b))
bernProb_sum_left	line 159: theorem bernProb_sum_left {beta : Type*} [DecidableEq beta] [Fintype beta] (p : Real) (A : (alpha -> Bool) -> Prop) : bernProb p (fun sigma : alpha + beta -> Bool => A (fun a : alpha => sigma (Sum.inl a))) = bernProb p A
bernProb_sum_inter	line 192: theorem bernProb_sum_inter {beta : Type*} [DecidableEq beta] [Fintype beta] (p : Real) (A : (alpha -> Bool) -> Prop) (B : (beta -> Bool) -> Prop) : bernProb p (fun sigma : alpha + beta -> Bool => A (fun a : alpha => sigma (Sum.inl a)) /\ B (fun b : beta => sigma (Sum.inr b))) = bernProb p A * bernProb p B
probOn_nonneg	line 289: theorem probOn_nonneg {d : Nat} {p : Real} {F : Finset (Bond d)} {A : (F -> Bool) -> Prop} (hp0 : 0 <= p) (hp1 : p <= 1) : 0 <= ProbOn p F A
probOn_le_one	line 294: theorem probOn_le_one {d : Nat} {p : Real} {F : Finset (Bond d)} {A : (F -> Bool) -> Prop} (hp0 : 0 <= p) (hp1 : p <= 1) : ProbOn p F A <= 1
probOn_univ	line 299: theorem probOn_univ {d : Nat} {p : Real} {F : Finset (Bond d)} (hp0 : 0 <= p) (hp1 : p <= 1) : ProbOn p F (fun _ => True) = 1
probOn_compl	line 304: theorem probOn_compl {d : Nat} {p : Real} {F : Finset (Bond d)} {A : (F -> Bool) -> Prop} (hp0 : 0 <= p) (hp1 : p <= 1) : ProbOn p F (fun sigma => not A sigma) = 1 - ProbOn p F A
probOn_mono	line 309: theorem probOn_mono {d : Nat} {p : Real} {F : Finset (Bond d)} {A B : (F -> Bool) -> Prop} (hAB : forall sigma, A sigma -> B sigma) (hp0 : 0 <= p) (hp1 : p <= 1) : ProbOn p F A <= ProbOn p F B
probOn_union_bound	line 315: theorem probOn_union_bound {d : Nat} {p : Real} {F : Finset (Bond d)} {A B : (F -> Bool) -> Prop} (hp0 : 0 <= p) (hp1 : p <= 1) : ProbOn p F (fun sigma => A sigma \/ B sigma) <= ProbOn p F A + ProbOn p F B

Sharpness/LocalEvent.lean

Imports. Sharpness.FiniteBernoulli.

Definitions, structures, and abbreviations.

Name	Statement or signature
DependsOn	line 8: def DependsOn {d : Nat} (F : Finset (Bond d)) (A : Config d -> Prop) : Prop -- A global event depends only on the finite support ‘F‘.
LocalEvent	line 13: structure LocalEvent (d : Nat) -- A local event is a global predicate together with an explicit finite bond support.
extendConfig [noncomputable]	line 19: noncomputable def extendConfig {d : Nat} (F : Finset (Bond d)) (sigma : F -> Bool) : Config d -- Extend an assignment on a finite support to a global configuration.
Prob [noncomputable]	line 24: noncomputable def Prob {d : Nat} (p : Real) (E : LocalEvent d) : Real -- Probability of a local event, computed on its explicit finite support.
supportExtensionEquiv [private, noncomputable]	line 27: private noncomputable def supportExtensionEquiv {d : Nat} (F G : Finset (Bond d)) (hsub : F <= G) : SubtypeF + Subtype(G \ F) ~= SubtypeG
LocalEvent.compl	line 56: def LocalEvent.compl {d : Nat} (E : LocalEvent d) : LocalEvent d -- Complement of a local event.
LocalEvent.inter	line 64: def LocalEvent.inter {d : Nat} (E F : LocalEvent d) : LocalEvent d -- Intersection of two local events.
LocalEvent.union	line 86: def LocalEvent.union {d : Nat} (E F : LocalEvent d) : LocalEvent d -- Union of two local events.

Theorems and lemmas.

Name	Statement or signature
supportExtensionEquiv_symm_left [private]	line 49: private theorem supportExtensionEquiv_symm_left {d : Nat} (F G : Finset (Bond d)) (hsub : F <= G) {e : Bond d} (he : e in F) : (supportExtensionEquiv F G hsub).symm <e, hsub he> = Sum.inl <e, he>
prob_support_mono	line 113: theorem prob_support_mono {d : Nat} {p : Real} (E : LocalEvent d) {G : Finset (Bond d)} (hsub : E.support <= G) : Prob p E = ProbOn p G (fun sigma => E.pred (extendConfig G sigma))

Sharpness/Independence.lean

Imports. Sharpness.LocalEvent.

Definitions, structures, and abbreviations. *None.*

Theorems and lemmas.

Name	Statement or signature
prob_inter_eq_mul_of_disjoint	line 5: theorem prob_inter_eq_mul_of_disjoint {d : Nat} {p : Real} (E F : LocalEvent d) (hdisj : Disjoint E.support F.support) (hp0 : 0 <= p) (hp1 : p <= 1) : Prob p (E.inter F) = Prob p E * Prob p F
prob_inter_three_eq_mul_of_pairwise_disjoint	line 65: theorem prob_inter_three_eq_mul_of_pairwise_disjoint {d : Nat} {p : Real} (E F G : LocalEvent d) (hEF : Disjoint E.support F.support) (hEG : Disjoint E.support G.support) (hFG : Disjoint F.support G.support) (hp0 : 0 <= p) (hp1 : p <= 1) : Prob p ((E.inter F).inter G) = Prob p E * Prob p F * Prob p G

Sharpness/Monotonicity.lean

Imports. Sharpness.LocalEvent.

Definitions, structures, and abbreviations.

Name	Statement or signature
ConfigLE	line 8: def ConfigLE {d : Nat} (omega omega' : Config d) : Prop -- Coordinatewise partial order on configurations.
Increasing	line 12: def Increasing {d : Nat} (E : LocalEvent d) : Prop -- A local event is increasing if opening more bonds preserves occurrence.
BoolAssignmentLE	line 16: def BoolAssignmentLE {alpha : Type*} (sigma tau : alpha -> Bool) : Prop -- Coordinatewise partial order on finite Boolean assignments.
IncreasingBoolEvent	line 20: def IncreasingBoolEvent {alpha : Type*} (A : (alpha -> Bool) -> Prop) : Prop -- Increasing event on a finite Boolean cube.

Theorems and lemmas.

Name	Statement or signature
bernWeight_fin_cons [private]	line 23: private theorem bernWeight_fin_cons {n : Nat} (p : Real) (b : Bool) (tau : Fin n -> Bool) : bernWeight p (Fin.cons b tau) = (if b then p else 1 - p) * bernWeight p tau
bernProb_fin_succ [private]	line 35: private theorem bernProb_fin_succ {n : Nat} (p : Real) (A : (Fin (n + 1) -> Bool) -> Prop) : bernProb p A = (1 - p) * bernProb p (fun tau : Fin n -> Bool => A (Fin.cons false tau)) + p * bernProb p (fun tau : Fin n -> Bool => A (Fin.cons true tau))
IncreasingBoolEvent_fin_tail_false [private]	line 71: private theorem IncreasingBoolEvent_fin_tail_false {n : Nat} {A : (Fin (n + 1) -> Bool) -> Prop} (hinc : IncreasingBoolEvent A) : IncreasingBoolEvent (fun tau : Fin n -> Bool => A (Fin.cons false tau))
IncreasingBoolEvent_fin_tail_true [private]	line 81: private theorem IncreasingBoolEvent_fin_tail_true {n : Nat} {A : (Fin (n + 1) -> Bool) -> Prop} (hinc : IncreasingBoolEvent A) : IncreasingBoolEvent (fun tau : Fin n -> Bool => A (Fin.cons true tau))
fin_false_le_true [private]	line 91: private theorem fin_false_le_true {n : Nat} (tau : Fin n -> Bool) : BoolAssignmentLE (Fin.cons false tau) (Fin.cons true tau)

Name	Statement or signature
bernProb_fin_mono_parameter [private]	line 98: private theorem bernProb_fin_mono_parameter (n : Nat) {p q : Real} {A : (Fin n -> Bool) -> Prop} (hinc : IncreasingBoolEvent A) (hp0 : 0 <= p) (hpq : p <= q) (hq1 : q <= 1) : bernProb p A <= bernProb q A
bernProb_mono_parameter	line 138: theorem bernProb_mono_parameter {alpha : Type*} [DecidableEq alpha] [Fintype alpha] {p q : Real} {A : (alpha -> Bool) -> Prop} (hinc : IncreasingBoolEvent A) (hp0 : 0 <= p) (hpq : p <= q) (hq1 : q <= 1) : bernProb p A <= bernProb q A
extendConfig_mono	line 189: theorem extendConfig_mono {d : Nat} {F : Finset (Bond d)} {sigma tau : F -> Bool} (hle : BoolAssignmentLE sigma tau) : ConfigLE (extendConfig F sigma) (extendConfig F tau)
prob_mono	line 198: theorem prob_mono {d : Nat} {p q : Real} (E : LocalEvent d) (hinc : Increasing E) (hp0 : 0 <= p) (hpq : p <= q) (hq1 : q <= 1) : Prob p E <= Prob q E

Sharpness/Russo.lean

Imports. Sharpness.Independence, Sharpness.Monotonicity.

Definitions, structures, and abbreviations.

Name	Statement or signature
indic [noncomputable]	line 13: noncomputable def indic (P : Prop) (x : Real) : Real -- Real-valued indicator, with classical decidability kept out of theorem statements.
forceBool	line 18: def forceBool (e : alpha) (b : Bool) (sigma : alpha -> Bool) : alpha -> Bool -- Force one Boolean coordinate to a chosen value.
ClosedPivotalBool	line 22: def ClosedPivotalBool (A : (alpha -> Bool) -> Prop) (e : alpha) (sigma : alpha -> Bool) : Prop -- Closed-pivotality on a finite Boolean cube.
configAt	line 27: def configAt (e : alpha) (b : Bool) (tau : {f : alpha // f != e} -> Bool) : alpha -> Bool -- Rebuild a configuration from one coordinate and an assignment of the other coordinates.
deleteCoord	line 31: def deleteCoord (e : alpha) (sigma : alpha -> Bool) : {f : alpha // f != e} -> Bool -- Delete one coordinate from an assignment.
boolFactor	line 64: def boolFactor (sigma : alpha -> Bool) (e : alpha) (q : Real) : Real -- The one-site Bernoulli factor.
bernWeightScoreCoord [noncomputable]	line 68: noncomputable def bernWeightScoreCoord (p : Real) (sigma : alpha -> Bool) (e : alpha) : Real -- Derivative contribution of one coordinate in one configuration.
splitAt	line 168: def splitAt (e : alpha) : (alpha -> Bool) ~= Bool x ({f : alpha // f != e} -> Bool) -- Split a Boolean assignment into the distinguished coordinate and the other coordinates.
forceOpen	line 364: def forceOpen {d : Nat} (e : Bond d) (omega : Config d) : Config d -- Force a bond to be open in a configuration.
ClosedPivotal	line 368: def ClosedPivotal {d : Nat} (E : LocalEvent d) (e : Bond d) (omega : Config d) : Prop -- Closed-pivotal event for an increasing local event.
closedPivotalEvent	line 399: def closedPivotalEvent {d : Nat} (E : LocalEvent d) (e : Bond d) : LocalEvent d -- Local event that a fixed bond is closed-pivotal.

Theorems and lemmas.

Name	Statement or signature
configAt_self	line 35: lemma configAt_self (e : alpha) (b : Bool) (tau : {f : alpha // f != e} -> Bool) : configAt e b tau e = b
configAt_ne	line 41: lemma configAt_ne (e f : alpha) (hfe : f != e) (b : Bool) (tau : {f : alpha // f != e} -> Bool) : configAt e b tau f = tau <f, hfe>
deleteCoord_configAt	line 47: lemma deleteCoord_configAt (e : alpha) (b : Bool) (tau : {f : alpha // f != e} -> Bool) : deleteCoord e (configAt e b tau) = tau
forceBool_configAt_false	line 54: lemma forceBool_configAt_false (e : alpha) (tau : {f : alpha // f != e} -> Bool) : forceBool e true (configAt e false tau) = configAt e true tau

Name	Statement or signature
erase_prod_boolFactor_eq_deleteCoord	line 72: lemma erase_prod_boolFactor_eq_deleteCoord (p : Real) (sigma : alpha -> Bool) (e : alpha) : ((Finset.univ.erase e).prod fun f : alpha => boolFactor sigma f p) = bernWeight p (deleteCoord e sigma)
bernWeight_hasDerivAt_score	line 87: lemma bernWeight_hasDerivAt_score (sigma : alpha -> Bool) (p : Real) : HasDerivAt (fun q : Real => bernWeight q sigma) (Finset.univ.sum fun e : alpha => bernWeightScoreCoord p sigma e) p
bernWeight_configAt_false	line 115: lemma bernWeight_configAt_false (p : Real) (e : alpha) (tau : {f : alpha // f != e} -> Bool) : bernWeight p (configAt e false tau) = (1 - p) * bernWeight p tau
bernWeight_configAt_true	line 136: lemma bernWeight_configAt_true (p : Real) (e : alpha) (tau : {f : alpha // f != e} -> Bool) : bernWeight p (configAt e true tau) = p * bernWeight p tau
scoreCoord_configAt_false	line 157: lemma scoreCoord_configAt_false (p : Real) (e : alpha) (tau : {f : alpha // f != e} -> Bool) : bernWeightScoreCoord p (configAt e false tau) e = -bernWeight p tau
scoreCoord_configAt_true	line 162: lemma scoreCoord_configAt_true (p : Real) (e : alpha) (tau : {f : alpha // f != e} -> Bool) : bernWeightScoreCoord p (configAt e true tau) e = bernWeight p tau
score_pair_eq_pivotal	line 184: lemma score_pair_eq_pivotal (A : (alpha -> Bool) -> Prop) (hinc : IncreasingBoolEvent A) (p : Real) (e : alpha) (tau : {f : alpha // f != e} -> Bool) : indic (A (configAt e false tau)) (bernWeightScoreCoord p (configAt e false tau) e) + indic (A (configAt e true tau)) (bernWeightScoreCoord p (configAt e true tau) e) = indic (ClosedPivotalBool A e (configAt e false tau)) (bernWeight p tau)
closedPivotal_configAt_true_false	line 216: lemma closedPivotal_configAt_true_false (A : (alpha -> Bool) -> Prop) (e : alpha) (tau : {f : alpha // f != e} -> Bool) : not ClosedPivotalBool A e (configAt e true tau)
closedPivotal_prob_pair	line 222: lemma closedPivotal_prob_pair (A : (alpha -> Bool) -> Prop) (p : Real) (e : alpha) (tau : {f : alpha // f != e} -> Bool) : indic (ClosedPivotalBool A e (configAt e false tau)) (bernWeight p (configAt e false tau)) + indic (ClosedPivotalBool A e (configAt e true tau)) (bernWeight p (configAt e true tau)) = (1 - p) * indic (ClosedPivotalBool A e (configAt e false tau)) (bernWeight p tau)
scoreCoord_sum_eq_closedPivotal	line 237: lemma scoreCoord_sum_eq_closedPivotal (A : (alpha -> Bool) -> Prop) (hinc : IncreasingBoolEvent A) {p : Real} (hp1 : p < 1) (e : alpha) : (Finset.univ.sum fun sigma : alpha -> Bool => indic (A sigma) (bernWeightScoreCoord p sigma e)) = (1 / (1 - p)) * bernProb p (ClosedPivotalBool A e)
indic_sum	line 306: lemma indic_sum (P : Prop) (f : alpha -> Real) : indic P (Finset.univ.sum f) = Finset.univ.sum fun e : alpha => indic P (f e)
bernProb_hasDerivAt_score	line 311: lemma bernProb_hasDerivAt_score (A : (alpha -> Bool) -> Prop) (p : Real) : HasDerivAt (fun q : Real => bernProb q A) (Finset.univ.sum fun sigma : alpha -> Bool => indic (A sigma) (Finset.univ.sum fun e : alpha => bernWeightScoreCoord p sigma e)) p
score_total_eq_closedPivotal	line 324: lemma score_total_eq_closedPivotal (A : (alpha -> Bool) -> Prop) (hinc : IncreasingBoolEvent A) {p : Real} (hp1 : p < 1) : (Finset.univ.sum fun sigma : alpha -> Bool => indic (A sigma) (Finset.univ.sum fun e : alpha => bernWeightScoreCoord p sigma e)) = (1 / (1 - p)) * (Finset.univ.sum fun e : alpha => bernProb p (ClosedPivotalBool A e))
bernProb_russo_closed_pivotal	line 353: theorem bernProb_russo_closed_pivotal (A : (alpha -> Bool) -> Prop) (hinc : IncreasingBoolEvent A) {p : Real} (_hp0 : 0 < p) (hp1 : p < 1) : HasDerivAt (fun q : Real => bernProb q A) ((1 / (1 - p)) * (Finset.univ.sum fun e : alpha => bernProb p (ClosedPivotalBool A e))) p -- Russo's formula on a finite Boolean product in closed-pivotal form.
closedPivotal_dependsOn	line 372: theorem closedPivotal_dependsOn {d : Nat} (E : LocalEvent d) (e : Bond d) : DependsOn (insert e E.support) (ClosedPivotal E e)
closedPivotal_restrict_iff	line 404: lemma closedPivotal_restrict_iff {d : Nat} (E : LocalEvent d) {e : Bond d} (he : e in E.support) (sigma : E.support -> Bool) : ClosedPivotal E e (extendConfig E.support sigma) <-> ClosedPivotalBool (fun sigma : E.support -> Bool => E.pred (extendConfig E. support sigma)) <e, he> sigma
closedPivotal_prob_restrict	line 433: lemma closedPivotal_prob_restrict {d : Nat} (E : LocalEvent d) {p : Real} (e : E.support) : bernProb p (ClosedPivotalBool (fun sigma : E.support -> Bool => E.pred (extendConfig E.support sigma)) e) = Prob p (closedPivotalEvent E e.1)
russo_closed_pivotal	line 460: theorem russo_closed_pivotal {d : Nat} (E : LocalEvent d) (hinc : Increasing E) {p : Real} (hp0 : 0 < p) (hp1 : p < 1) : HasDerivAt (fun q => Prob q E) ((1 / (1 - p)) * (E.support.sum fun e => Prob p (closedPivotalEvent E e))) p -- Russo's formula in closed-pivotal form for finite local increasing events.

Sharpness/Events.lean

Imports. Sharpness.LocalEvent, Sharpness.Monotonicity, Sharpness.Paths.

Definitions, structures, and abbreviations.

Name	Statement or signature
ExitPred	line 11: def ExitPred {d : Nat} (Lam : Finset (Vertex d)) (omega : Config d) : Prop -- Predicate for the finite-volume exit event from ‘Lam‘.
connInEvent [noncomputable]	line 47: noncomputable def connInEvent {d : Nat} (S : Finset (Vertex d)) (x y : Vertex d) : LocalEvent d -- Local event for ‘x‘ connected to ‘y‘ inside ‘S‘.
exitEvent [noncomputable]	line 99: noncomputable def exitEvent {d : Nat} (Lam : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) : LocalEvent d -- Local event that ‘0‘ exits the finite vertex set ‘Lam‘.
exitProb [noncomputable]	line 118: noncomputable def exitProb {d : Nat} (p : Real) (Lam : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) : Real -- Finite Bernoulli probability of the finite exit event from ‘Lam‘.
boxExitProb [noncomputable]	line 123: noncomputable def boxExitProb (d : Nat) (p : Real) (n : Nat) : Real -- Finite box exit probability ‘P_p(0 <-> Lambda_n^c)‘.
ConnToSetIn	line 127: def ConnToSetIn {d : Nat} (omega : Config d) (T : Finset (Vertex d)) (u : Vertex d) (B : Finset (Vertex d)) : Prop -- Connectivity from a vertex to a finite target set inside a finite ambient set.
connToSetInEvent [noncomputable]	line 143: noncomputable def connToSetInEvent {d : Nat} (T : Finset (Vertex d)) (u : Vertex d) (B : Finset (Vertex d)) : LocalEvent d -- Local event that ‘u‘ connects to the finite target set ‘B‘ inside ‘T‘.
theta [noncomputable]	line 166: noncomputable def theta (d : Nat) (p : Real) : Real -- Infinite-cluster density defined as the decreasing-limit infimum of finite exit probabilities.

Theorems and lemmas.

Name	Statement or signature
openPath_dependsOn_of_pathIn	line 16: theorem openPath_dependsOn_of_pathIn {d : Nat} {S : Finset (Vertex d)} {gamma : List (Vertex d)} {omega omega' : Config d} (hsame : forall e, e in internalBonds S -> omega e = omega' e) (hS : PathIn S gamma) : OpenPath omega gamma <-> OpenPath omega' gamma
connIn_dependsOn	line 37: theorem connIn_dependsOn {d : Nat} (S : Finset (Vertex d)) (x y : Vertex d) : DependsOn (internalBonds S) (fun omega => ConnIn omega S x y) -- Finite-set connectivity depends only on bonds internal to the finite set.
openPath_mono	line 53: theorem openPath_mono {d : Nat} {omega omega' : Config d} {gamma : List (Vertex d)} (hle : ConfigLE omega omega') (hgamma : OpenPath omega gamma) : OpenPath omega' gamma
connIn_mono	line 62: theorem connIn_mono {d : Nat} {omega omega' : Config d} {S : Finset (Vertex d)} {x y : Vertex d} (hle : ConfigLE omega omega') (hconn : ConnIn omega S x y) : ConnIn omega' S x y
connInEvent_increasing	line 68: theorem connInEvent_increasing {d : Nat} (S : Finset (Vertex d)) (x y : Vertex d) : Increasing (connInEvent S x y)
exitEvent_dependsOn	line 74: theorem exitEvent_dependsOn {d : Nat} (Lam : Finset (Vertex d)) (_h0 : (0 : Vertex d) in Lam) : DependsOn (internalBonds Lam union exitBonds Lam) (ExitPred Lam) -- The finite exit event is local on internal and boundary bonds of ‘Lam‘.
exitPred_mono	line 105: theorem exitPred_mono {d : Nat} {Lam : Finset (Vertex d)} {omega omega' : Config d} (hle : ConfigLE omega omega') (hexit : ExitPred Lam omega) : ExitPred Lam omega'
exitEvent_increasing	line 112: theorem exitEvent_increasing {d : Nat} (Lam : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) : Increasing (exitEvent Lam h0)
connToSetIn_dependsOn	line 132: theorem connToSetIn_dependsOn {d : Nat} (T : Finset (Vertex d)) (u : Vertex d) (B : Finset (Vertex d)) : DependsOn (internalBonds T) (fun omega => ConnToSetIn omega T u B) -- Finite target connectivity is local on the ambient internal bonds.
connToSetIn_mono	line 149: theorem connToSetIn_mono {d : Nat} {omega omega' : Config d} {T : Finset (Vertex d)} {u : Vertex d} {B : Finset (Vertex d)} (hle : ConfigLE omega omega') (hconn : ConnToSetIn omega T u B) : ConnToSetIn omega' T u B
connToSetInEvent_increasing	line 156: theorem connToSetInEvent_increasing {d : Nat} (T : Finset (Vertex d)) (u : Vertex d) (B : Finset (Vertex d)) : Increasing (connToSetInEvent T u B)

Name	Statement or signature
boxExitProb_nonneg	line 169: theorem boxExitProb_nonneg {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (n : Nat) : 0 <= boxExitProb d p n
theta_nonneg	line 178: theorem theta_nonneg {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) : 0 <= theta d p
theta_le_boxExitProb	line 186: theorem theta_le_boxExitProb (d : Nat) (p : Real) (n : Nat) (hp0 : 0 <= p) (hp1 : p <= 1) : theta d p <= boxExitProb d p n
theta_eq_zero_of_exponential_decay	line 196: theorem theta_eq_zero_of_exponential_decay {d : Nat} {p c : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (hc : 0 < c) (hdecay : forall n : Nat, boxExitProb d p n <= Real.exp (-(c * (n : Real)))) : theta d p = 0
le_theta_of_le_boxExitProb	line 219: theorem le_theta_of_le_boxExitProb {d : Nat} {p b : Real} (hb : forall n : Nat, b <= boxExitProb d p n) : b <= theta d p

Sharpness/Phi.lean

Imports. Sharpness.Events, Sharpness.Monotonicity.

Definitions, structures, and abbreviations.

Name	Statement or signature
phi [noncomputable]	line 9: noncomputable def phi {d : Nat} (p : Real) (S : Finset (Vertex d)) : Real -- The finite-set criterion quantity ‘phi_p(S)‘.
finiteSubsetsWithZero [noncomputable]	line 13: noncomputable def finiteSubsetsWithZero {d : Nat} (Lam : Finset (Vertex d)) : Finset (Finset (Vertex d)) -- Finite subsets of ‘Lam‘ that contain the origin.
phiMinIn [noncomputable]	line 18: noncomputable def phiMinIn {d : Nat} (p : Real) (Lam : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) : Real -- Finite minimum of ‘phi_p(S)‘ over ‘S subset Lam‘ with ‘0 in S‘.
pTildeSet [noncomputable]	line 26: noncomputable def pTildeSet (d : Nat) : Set Real -- Parameters satisfying the finite-set criterion.
pTilde [noncomputable]	line 31: noncomputable def pTilde (d : Nat) : Real -- The finite-set critical point ‘pTilde‘.

Theorems and lemmas.

Name	Statement or signature
zero_mem_pTildeSet	line 35: theorem zero_mem_pTildeSet (d : Nat) : (0 : Real) in pTildeSet d -- ‘0‘ belongs to the defining set of ‘pTilde‘.
pTildeSet_nonempty	line 42: theorem pTildeSet_nonempty (d : Nat) : (pTildeSet d).Nonempty -- The defining set of ‘pTilde‘ is nonempty.
pTildeSet_bddAbove	line 46: theorem pTildeSet_bddAbove (d : Nat) : BddAbove (pTildeSet d) -- The defining set of ‘pTilde‘ is bounded above by ‘1‘.
zero_le_pTilde	line 52: theorem zero_le_pTilde (d : Nat) : 0 <= pTilde d -- The finite-set critical point is nonnegative.
pTilde_le_one	line 56: theorem pTilde_le_one (d : Nat) : pTilde d <= 1 -- The finite-set critical point is at most one.
phi_mono	line 60: theorem phi_mono {d : Nat} {p q : Real} (S : Finset (Vertex d)) (hp0 : 0 <= p) (hpq : p <= q) (hq1 : q <= 1) : phi p S <= phi q S -- For each finite ‘S‘, ‘phi_p(S)‘ is monotone in the Bernoulli parameter on ‘[0,1]‘.
exists_phi_lt_one_of_lt_pTilde	line 91: theorem exists_phi_lt_one_of_lt_pTilde {d : Nat} {p : Real} (hp0 : 0 <= p) (hp : p < pTilde d) : exists S : Finset (Vertex d), (0 : Vertex d) in S /\ phi p S < 1 -- Below ‘pTilde‘, some finite set satisfies the finite-set criterion.
one_le_phi_of_pTilde_lt	line 101: theorem one_le_phi_of_pTilde_lt {d : Nat} {p : Real} (hp : pTilde d < p) (hp1 : p <= 1) (S : Finset (Vertex d)) (h0 : (0 : Vertex d) in S) : 1 <= phi p S -- Above ‘pTilde‘, every finite set containing the origin violates the finite-set criterion.

Sharpness/DiffIneq.lean

Imports. Sharpness.Phi, Sharpness.Russo.

Definitions, structures, and abbreviations.

Name	Statement or signature
ExitFromPred	line 9: def ExitFromPred {d : Nat} (Lam : Finset (Vertex d)) (x : Vertex d) (omega : Config d) : Prop -- Finite-volume exit from a vertex ‘x‘ through the oriented boundary of ‘Lam‘.
Shield [noncomputable]	line 46: noncomputable def Shield {d : Nat} (Lam : Finset (Vertex d)) (omega : Config d) : Finset (Vertex d) -- The random shield set ‘{x in Lam | x‘ is not connected to ‘Lam^c‘}.
closeInternalEdges [noncomputable]	line 58: noncomputable def closeInternalEdges {d : Nat} (S : Finset (Vertex d)) (omega : Config d) : Config d -- Closing all internal edges of ‘S‘.
shieldSupport [noncomputable]	line 63: noncomputable def shieldSupport {d : Nat} (Lam S : Finset (Vertex d)) : Finset (Bond d) -- The finite support for ‘{Shield = S}‘ after deleting internal ‘S‘-edges.
shieldEvent [noncomputable]	line 455: noncomputable def shieldEvent {d : Nat} (Lam S : Finset (Vertex d)) (hS : S <= Lam) : LocalEvent d -- Local event for a fixed shield value, using the separated support.
finsetUnionEvent [private, noncomputable]	line 882: private noncomputable def finsetUnionEvent {d : Nat} {alpha : Type*} (s : Finset alpha) (E : alpha -> LocalEvent d) : LocalEvent d

Theorems and lemmas.

Name	Statement or signature
exitFrom_zero_iff_exitPred	line 16: theorem exitFrom_zero_iff_exitPred {d : Nat} (Lam : Finset (Vertex d)) (omega : Config d) : ExitFromPred Lam (0 : Vertex d) omega <-> ExitPred Lam omega -- Exit from the origin is the finite exit predicate used by ‘exitEvent‘.
exitFrom_dependsOn	line 22: theorem exitFrom_dependsOn {d : Nat} (Lam : Finset (Vertex d)) (x : Vertex d) : DependsOn (internalBonds Lam union exitBonds Lam) (ExitFromPred Lam x) -- Exit from a fixed vertex is local on the finite exit support of ‘Lam‘.
zero_mem_shield_iff_not_exit	line 52: theorem zero_mem_shield_iff_not_exit {d : Nat} (Lam : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) (omega : Config d) : (0 : Vertex d) in Shield Lam omega <-> not ExitPred Lam omega -- The origin belongs to the shield exactly when the finite exit event fails.
bondOfAdj_mem_internalBonds_endpoints [private]	line 67: private theorem bondOfAdj_mem_internalBonds_endpoints {d : Nat} {S : Finset (Vertex d)} {x y : Vertex d} {hxy : Adj x y} (hb : bondOfAdj hxy in internalBonds S) : x in S /\ y in S
bondOfAdj_not_mem_internalBonds_of_not_both [private]	line 101: private theorem bondOfAdj_not_mem_internalBonds_of_not_both {d : Nat} {S : Finset (Vertex d)} {x y : Vertex d} (hxy : Adj x y) (hnot : not (x in S /\ y in S)) : bondOfAdj hxy notin internalBonds S
openPath_of_closeInternalEdges [private]	line 108: private theorem openPath_of_closeInternalEdges {d : Nat} {S : Finset (Vertex d)} {omega : Config d} {gamma : List (Vertex d)} (hopen : OpenPath (closeInternalEdges S omega) gamma) : OpenPath omega gamma
connIn_of_closeInternalEdges [private]	line 125: private theorem connIn_of_closeInternalEdges {d : Nat} {S Lam : Finset (Vertex d)} {omega : Config d} {x y : Vertex d} (hconn : ConnIn (closeInternalEdges S omega) Lam x y) : ConnIn omega Lam x y
exitFrom_of_closeInternalEdges [private]	line 132: private theorem exitFrom_of_closeInternalEdges {d : Nat} {S Lam : Finset (Vertex d)} {omega : Config d} {x : Vertex d} (hexit : ExitFromPred Lam x (closeInternalEdges S omega)) : ExitFromPred Lam x omega
connIn_single [private]	line 142: private theorem connIn_single {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {x : Vertex d} (hx : x in S) : ConnIn omega S x x
openPath_closeInternalEdges_of_no_internal [private]	line 151: private theorem openPath_closeInternalEdges_of_no_internal {d : Nat} {S : Finset (Vertex d)} {omega : Config d} {gamma : List (Vertex d)} (hopen : OpenPath omega gamma) (hnotInternal : forall {x y : Vertex d}, x in gamma -> y in gamma -> forall hxy : Adj x y, bondOfAdj hxy notin internalBonds S) : OpenPath (closeInternalEdges S omega) gamma
openPath_closeInternalEdges_of_unique_S [private]	line 182: private theorem openPath_closeInternalEdges_of_unique_S {d : Nat} {S : Finset (Vertex d)} {omega : Config d} {gamma : List (Vertex d)} {u : Vertex d} (hopen : OpenPath omega gamma) (hunique : forall z, z in gamma -> z in S -> z = u) : OpenPath (closeInternalEdges S omega) gamma
exists_closed_conn_from_last_S [private]	line 195: private theorem exists_closed_conn_from_last_S {d : Nat} {S Lam : Finset (Vertex d)} {omega : Config d} : forall {gamma : List (Vertex d)} {x y : Vertex d}, PathFromTo gamma x y -> PathIn Lam gamma -> OpenPath omega gamma -> (exists z, z in gamma /\ z in S) -> y notin S -> exists u, u in S /\ ConnIn (closeInternalEdges S omega) Lam u y

Name	Statement or signature
exists_exitFrom_close_of_exitFrom_from_S [private]	line 252: private theorem exists_exitFrom_close_of_exitFrom_from_S {d : Nat} {Lam S : Finset (Vertex d)} (hS : S <= Lam) {omega : Config d} {x : Vertex d} (hxS : x in S) (hexit : ExitFromPred Lam x omega) : exists u, u in S /\ ExitFromPred Lam u (closeInternalEdges S omega)
exitFrom_close_of_exitFrom_of_no_S_exit [private]	line 279: private theorem exitFrom_close_of_exitFrom_of_no_S_exit {d : Nat} {Lam S : Finset (Vertex d)} (hS : S <= Lam) {omega : Config d} {x : Vertex d} (hnoSExit : forall z, z in S -> not ExitFromPred Lam z omega) (hexit : ExitFromPred Lam x omega) : ExitFromPred Lam x (closeInternalEdges S omega)
disjoint_internalBonds_shieldSupport	line 322: theorem disjoint_internalBonds_shieldSupport {d : Nat} (Lam S : Finset (Vertex d)) : Disjoint (internalBonds S) (shieldSupport Lam S) -- Internal ‘S‘-bonds are disjoint from the support used by the shield event.
shield_dependsOn_ambient	line 330: theorem shield_dependsOn_ambient {d : Nat} (Lam S : Finset (Vertex d)) : DependsOn (internalBonds Lam union exitBonds Lam) (fun omega => Shield Lam omega = S) -- The shield value is local on the ambient finite exit support.
shield_eq_iff_deleted_internal	line 354: theorem shield_eq_iff_deleted_internal {d : Nat} (Lam S : Finset (Vertex d)) (hS : S <= Lam) (omega : Config d) : Shield Lam omega = S <-> ((forall x, x in S -> not ExitFromPred Lam x (closeInternalEdges S omega)) /\ (forall x, x in Lam -> x notin S -> ExitFromPred Lam x (closeInternalEdges S omega))) -- Deleted-internal-edge characterization of ‘{Shield = S}‘. After closing all internal ‘S‘-edges, shield equality is equivalent to no ‘S‘-vertex exiting and every ‘Lam \ S‘ vertex exiting.
shield_dependsOn_noninternal	line 416: theorem shield_dependsOn_noninternal {d : Nat} (Lam S : Finset (Vertex d)) (hS : S <= Lam) : DependsOn (shieldSupport Lam S) (fun omega => Shield Lam omega = S) -- Support separation for the shield event ‘{Shield = S}‘. This combines ‘shield_eq_iff_deleted_internal‘ with locality of finite exit events in the deleted configuration to show that internal ‘S‘-edges are irrelevant.
connIn_subset [private]	line 461: private theorem connIn_subset {d : Nat} {omega : Config d} {S T : Finset (Vertex d)} {x y : Vertex d} (hST : S <= T) (hconn : ConnIn omega S x y) : ConnIn omega T x y
connIn_append_open_adj_same [private]	line 468: private theorem connIn_append_open_adj_same {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {x y z : Vertex d} (hzS : z in S) (hyz : Adj y z) (hopen : omega (bondOfAdj hyz) = true) (hconn : ConnIn omega S x y) : ConnIn omega S x z
connIn_cons_open_adj [private]	line 505: private theorem connIn_cons_open_adj {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {x y z : Vertex d} (hxS : x in S) (hxy : Adj x y) (hopen : omega (bondOfAdj hxy) = true) (hconn : ConnIn omega S y z) : ConnIn omega S x z
connIn_trans [private]	line 531: private theorem connIn_trans {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {x y z : Vertex d} (hxy : ConnIn omega S x y) (hyz : ConnIn omega S y z) : ConnIn omega S x z
connIn_forceOpen_of_connIn [private]	line 582: private theorem connIn_forceOpen_of_connIn {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {x y : Vertex d} {b : Bond d} (hconn : ConnIn omega S x y) : ConnIn (forceOpen b omega) S x y
connIn_of_forceOpen_of_not_internal [private]	line 594: private theorem connIn_of_forceOpen_of_not_internal {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {x y : Vertex d} {b : Bond d} (hb : b notin internalBonds S) (hconn : ConnIn (forceOpen b omega) S x y) : ConnIn omega S x y
first_exit_conn_of_path [private]	line 606: private theorem first_exit_conn_of_path {d : Nat} {omega : Config d} {S : Finset (Vertex d)} : forall {gamma : List (Vertex d)} {x y : Vertex d}, PathFromTo gamma x y -> OpenPath omega gamma -> x in S -> y notin S -> exists a c : Vertex d, exists hac : Adj a c, ConnIn omega S x a /\ a in S /\ c notin S /\ omega (bondOfAdj hac) = true
exitFrom_mono [private]	line 648: private theorem exitFrom_mono {d : Nat} {omega omega' : Config d} {Lam : Finset (Vertex d)} {x : Vertex d} (hle : ConfigLE omega omega') (hexit : ExitFromPred Lam x omega) : ExitFromPred Lam x omega'
exitPred_of_connIn_boundary_open [private]	line 656: private theorem exitPred_of_connIn_boundary_open {d : Nat} (Lam S : Finset (Vertex d)) {omega : Config d} {e : OrientedEdge d} (hSLam : S <= Lam) (hout : forall y, y in Lam -> y notin S -> ExitFromPred Lam y omega) (he : e in orientedBoundary S) (hconn : ConnIn omega S (0 : Vertex d) e.1) (hopen : omega (bondOfAdj (orientedBoundary_adj he)) = true) : ExitPred Lam omega
exitPred_of_connIn_shield_boundary_open [private]	line 685: private theorem exitPred_of_connIn_shield_boundary_open {d : Nat} (Lam : Finset (Vertex d)) {omega : Config d} {e : OrientedEdge d} (he : e in orientedBoundary (Shield Lam omega)) (hconn : ConnIn omega (Shield Lam omega) (0 : Vertex d) e.1) (hopen : omega (bondOfAdj (orientedBoundary_adj he)) = true) : ExitPred Lam omega

Name	Statement or signature
boundary_left_eq_of_bond_eq [private]	line 701: private theorem boundary_left_eq_of_bond_eq {d : Nat} {S : Finset (Vertex d)} {a c : Vertex d} {hac : Adj a c} {e : OrientedEdge d} (he : e in orientedBoundary S) (haS : a in S) (hbond : bondOfAdj hac = bondOfAdj (orientedBoundary_adj he)) : a = e.1
closedPivotal_exit_iff_connIn_on_shield	line 723: theorem closedPivotal_exit_iff_connIn_on_shield {d : Nat} (Lam S : Finset (Vertex d)) (hS : S <= Lam) (h0S : (0 : Vertex d) in S) {e : OrientedEdge d} (he : e in orientedBoundary S) {omega : Config d} (hshield : Shield Lam omega = S) : ClosedPivotal (exitEvent Lam (hS h0S)) (bondOfAdj (orientedBoundary_adj he)) omega <-> ConnIn omega S (0 : Vertex d) e.1 -- Pivotal equivalence on a fixed shield value. On ‘{Shield = S}‘, an oriented boundary bond ‘(x,y) in boundary E S‘ is closed-pivotal for ‘A_Lam‘ iff ‘0‘ is connected to ‘x‘ inside ‘S‘.
phiMinIn_le_phi	line 828: theorem phiMinIn_le_phi {d : Nat} {p : Real} {Lam S : Finset (Vertex d)} (h0 : (0 : Vertex d) in Lam) (hS : S <= Lam) (h0S : (0 : Vertex d) in S) : phiMinIn p Lam h0 <= phi p S -- The finite minimum is below each admissible ‘phi p S‘.
prob_nonneg [private]	line 837: private theorem prob_nonneg {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (E : LocalEvent d) : 0 <= Prob p E
prob_le_of_pred_imp [private]	line 842: private theorem prob_le_of_pred_imp {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (E F : LocalEvent d) (himp : forall omega, E.pred omega -> F.pred omega) : Prob p E <= Prob p F
prob_union_le [private]	line 860: private theorem prob_union_le {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (E F : LocalEvent d) : Prob p (E.union F) <= Prob p E + Prob p F
prob_finsetUnionEvent_le_sum [private]	line 900: private theorem prob_finsetUnionEvent_le_sum {d : Nat} {alpha : Type*} [DecidableEq alpha] {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (s : Finset alpha) (E : alpha -> LocalEvent d) : Prob p (finsetUnionEvent s E) <= s.sum fun i => Prob p (E i)
sum_prob_le_prob_of_disjoint_subevents [private]	line 932: private theorem sum_prob_le_prob_of_disjoint_subevents {d : Nat} {alpha : Type*} [DecidableEq alpha] {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (s : Finset alpha) (E : LocalEvent d) (F : alpha -> LocalEvent d) (hdisj : forall i, i in s -> forall j, j in s -> i != j -> forall omega, (F i).pred omega -> (F j).pred omega -> False) (himp : forall i, i in s -> forall omega, (F i).pred omega -> E.pred omega) : s.sum (fun i => Prob p (F i)) <= Prob p E
phi_nonneg [private]	line 1023: private theorem phi_nonneg {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (S : Finset (Vertex d)) : 0 <= phi p S
phiMinIn_nonneg [private]	line 1032: private theorem phiMinIn_nonneg {d : Nat} {p : Real} (Lam : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) (hp0 : 0 <= p) (hp1 : p <= 1) : 0 <= phiMinIn p Lam h0
subset_of_mem_finiteSubsetsWithZero [private]	line 1041: private theorem subset_of_mem_finiteSubsetsWithZero {d : Nat} {Lam S : Finset (Vertex d)} (hS : S in finiteSubsetsWithZero Lam) : S <= Lam
zero_mem_of_mem_finiteSubsetsWithZero [private]	line 1047: private theorem zero_mem_of_mem_finiteSubsetsWithZero {d : Nat} {Lam S : Finset (Vertex d)} (hS : S in finiteSubsetsWithZero Lam) : (0 : Vertex d) in S
prob_exit_compl_eq [private]	line 1053: private theorem prob_exit_compl_eq {d : Nat} {p : Real} (Lam : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) (hp0 : 0 <= p) (hp1 : p <= 1) : Prob p ((exitEvent Lam h0).compl) = 1 - exitProb p Lam h0
one_sub_exitProb_le_sum_shield [private]	line 1059: private theorem one_sub_exitProb_le_sum_shield {d : Nat} {p : Real} (Lam : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) (hp0 : 0 <= p) (hp1 : p <= 1) : 1 - exitProb p Lam h0 <= (finiteSubsetsWithZero Lam).attach.sum (fun S => Prob p (shieldEvent Lam S.1 (subset_of_mem_finiteSubsetsWithZero S.2)))
bondOfAdj_mem_exitSupport_of_boundary_subset [private]	line 1097: private theorem bondOfAdj_mem_exitSupport_of_boundary_subset {d : Nat} {Lam S : Finset (Vertex d)} (hS : S <= Lam) (h0S : (0 : Vertex d) in S) {e : OrientedEdge d} (he : e in orientedBoundary S) : bondOfAdj (orientedBoundary_adj he) in (exitEvent Lam (hS h0S)). support
orientedBoundary_bond_injective [private]	line 1112: private theorem orientedBoundary_bond_injective {d : Nat} {S : Finset (Vertex d) } {e f : OrientedEdge d} (he : e in orientedBoundary S) (hf : f in orientedBoundary S) (hbond : bondOfAdj (orientedBoundary_adj he) = bondOfAdj (orientedBoundary_adj hf)) : e = f
conn_shield_prob_eq_mul [private]	line 1141: private theorem conn_shield_prob_eq_mul {d : Nat} {p : Real} (Lam S : Finset (Vertex d)) (hS : S <= Lam) (e : {e // e in orientedBoundary S}) (hp0 : 0 <= p) (hp1 : p <= 1) : Prob p ((connInEvent S (0 : Vertex d) e.1.1).inter (shieldEvent Lam S hS)) = Prob p (connInEvent S (0 : Vertex d) e.1.1) * Prob p (shieldEvent Lam S hS)

Name	Statement or signature
conn_shield_le_closedPivotal_shield [private]	line 1155: private theorem conn_shield_le_closedPivotal_shield {d : Nat} {p : Real} (Lam S : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) (hS : S <= Lam) (h0S : (0 : Vertex d) in S) (e : {e // e in orientedBoundary S}) (hp0 : 0 <= p) (hp1 : p <= 1) : Prob p ((connInEvent S (0 : Vertex d) e.1.1).inter (shieldEvent Lam S hS)) <= Prob p (((closedPivotalEvent (exitEvent Lam h0) (bondOfAdj (orientedBoundary_adj e.2))).inter (shieldEvent Lam S hS)))
shield_phiMin_div_mul_le_boundary_pivotal [private]	line 1176: private theorem shield_phiMin_div_mul_le_boundary_pivotal {d : Nat} {p : Real} (Lam S : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) (hS : S <= Lam) (h0S : (0 : Vertex d) in S) (hp0 : 0 < p) (hp1 : p <= 1) : (phiMinIn p Lam h0 / p) * Prob p (shieldEvent Lam S hS) <= (orientedBoundary S).attach.sum (fun e => Prob p (((closedPivotalEvent (exitEvent Lam h0) (bondOfAdj (orientedBoundary_adj e.2))).inter (shieldEvent Lam S hS))))
boundary_pivotal_shield_sum_le_support_sum [private]	line 1243: private theorem boundary_pivotal_shield_sum_le_support_sum {d : Nat} {p : Real} (Lam S : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) (hS : S <= Lam) (h0S : (0 : Vertex d) in S) (hp0 : 0 <= p) (hp1 : p <= 1) : (orientedBoundary S).attach.sum (fun e => Prob p (((closedPivotalEvent (exitEvent Lam h0) (bondOfAdj (orientedBoundary_adj e.2))).inter (shieldEvent Lam S hS)))) <= (exitEvent Lam h0).support.sum (fun b => Prob p (((closedPivotalEvent (exitEvent Lam h0) b).inter (shieldEvent Lam S hS))))
shield_phiMin_div_mul_le_support_pivotal [private]	line 1286: private theorem shield_phiMin_div_mul_le_support_pivotal {d : Nat} {p : Real} (Lam S : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) (hS : S <= Lam) (h0S : (0 : Vertex d) in S) (hp0 : 0 < p) (hp1 : p <= 1) : (phiMinIn p Lam h0 / p) * Prob p (shieldEvent Lam S hS) <= (exitEvent Lam h0).support.sum (fun b => Prob p (((closedPivotalEvent (exitEvent Lam h0) b).inter (shieldEvent Lam S hS))))
exit_russo_sum_lower_bound	line 1305: theorem exit_russo_sum_lower_bound {d : Nat} (Lam : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) {p : Real} (hp0 : 0 < p) (hp1 : p < 1) : (1 / (p * (1 - p))) * phiMinIn p Lam h0 * (1 - exitProb p Lam h0) <= (1 / (1 - p)) * ((exitEvent Lam h0).support. sum fun e => Prob p (closedPivotalEvent (exitEvent Lam h0) e)) -- The shield decomposition lower bound for the Russo closed-pivotal sum. The proof decomposes over finite shield values containing the origin, uses pivotal equivalence on each oriented shield boundary edge, applies the separated-support independence between internal ‘S‘ connections and ‘{Shield = S}‘, and then partitions the closed-pivotal events by the shield value.
differential_inequality	line 1381: theorem differential_inequality {d : Nat} (Lam : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) {p : Real} (hp0 : 0 < p) (hp1 : p < 1) : deriv (fun q => exitProb q Lam h0) p >= (1 / (p * (1 - p))) * phiMinIn p Lam h0 * (1 - exitProb p Lam h0) -- Lemma 2, the fundamental finite-volume differential inequality.

Sharpness/OdeComparison.lean

Imports. Mathlib.

Definitions, structures, and abbreviations. *None.*

Theorems and lemmas.

Name	Statement or signature
ode_log_comparison	line 12: theorem ode_log_comparison {f : Real -> Real} {q p : Real} (hq0 : 0 < q) (hqp : q < p) (hp1 : p < 1) (hderiv : forall t, q <= t -> t <= p -> HasDerivAt f (deriv f t) t) (hlt : forall t, q <= t -> t <= p -> f t < 1) (hineq : forall t, q < t -> t < p -> deriv f t >= (1 - f t) / (t * (1 - t))) : 1 - f p <= (1 - f q) * (q * (1 - p) / (p * (1 - q))) -- The real ODE/log comparison used in the supercritical argument. The differentiability hypothesis is stated on the closed interval. The endpoint regularity is needed mathematically: derivative bounds only on ‘(q, p)‘ do not control arbitrary jumps in the endpoint values of ‘f‘.

Sharpness/Supercritical.lean

Imports. Sharpness.DiffIneq, Sharpness.OdeComparison.

Definitions, structures, and abbreviations. *None.*

Theorems and lemmas.

Name	Statement or signature
bernWeight_all_false_pos [private]	line 8: private theorem bernWeight_all_false_pos {alpha : Type*} [Fintype alpha] {p : Real} (hp1 : p < 1) : 0 < bernWeight p (fun _ : alpha => false)
bernProb_pos_of_all_false [private]	line 17: private theorem bernProb_pos_of_all_false {alpha : Type*} [DecidableEq alpha] [Fintype alpha] {p : Real} {A : (alpha -> Bool) -> Prop} (hp0 : 0 <= p) (hp1 : p < 1) (hA : A (fun _ : alpha => false)) : 0 < bernProb p A
local_prob_lt_one_of_all_false_not [private]	line 40: private theorem local_prob_lt_one_of_all_false_not {d : Nat} {p : Real} (E : LocalEvent d) (hp0 : 0 <= p) (hp1 : p < 1) (hfalse : not E.pred (extendConfig E.support (fun _ : E.support => false))) : Prob p E < 1
exitProb_lt_one_of_lt_one	line 58: theorem exitProb_lt_one_of_lt_one {d : Nat} (Lam : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) {p : Real} (hp0 : 0 <= p) (hp1 : p < 1) : exitProb p Lam h0 < 1 -- Finite exit events have probability strictly below one when ‘p < 1‘: the all-closed assignment closes, in particular, every boundary edge.
one_le_phiMinIn_of_pTilde_lt [private]	line 70: private theorem one_le_phiMinIn_of_pTilde_lt {d : Nat} {t : Real} (Lam : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) (ht : pTilde d < t) (ht1 : t <= 1) : 1 <= phiMinIn t Lam h0
pTilde_endpoint_bound [private]	line 83: private theorem pTilde_endpoint_bound {a p x : Real} (ha0 : 0 <= a) (hap : a < p) (hp1 : p < 1) (hx : forall q, a < q -> q < p -> (p - q) / (p * (1 - q)) <= x) : (p - a) / (p * (1 - a)) <= x
finite_exit_lower_bound_above_q [private]	line 101: private theorem finite_exit_lower_bound_above_q {d : Nat} {q p : Real} (Lam : Finset (Vertex d)) (h0 : (0 : Vertex d) in Lam) (hptq : pTilde d < q) (hqp : q < p) (hp1 : p < 1) : (p - q) / (p * (1 - q)) <= exitProb p Lam h0
supercritical_lower_bound_above_pTilde	line 177: theorem supercritical_lower_bound_above_pTilde {d : Nat} {p : Real} (hp : pTilde d < p) (hp1 : p < 1) : theta d p >= (p - pTilde d) / (p * (1 - pTilde d)) -- Lemma 3, the lower bound above the finite-set critical point.

Sharpness/BoundaryIneq.lean

Imports. Sharpness.Clusters, Sharpness.Events, Sharpness.Independence.

Definitions, structures, and abbreviations.

Name	Statement or signature
finsetUnionEvent [private, noncomputable]	line 49: private noncomputable def finsetUnionEvent {d : Nat} {alpha : Type*} (s : Finset alpha) (E : alpha -> LocalEvent d) : LocalEvent d
clusterEvent [private, noncomputable]	line 99: private noncomputable def clusterEvent {d : Nat} (S C : Finset (Vertex d)) (u : Vertex d) : LocalEvent d
edgeOpenEvent [private, noncomputable]	line 105: private noncomputable def edgeOpenEvent {d : Nat} (b : Bond d) : LocalEvent d

Theorems and lemmas.

Name	Statement or signature
prob_le_of_pred_imp [private]	line 9: private theorem prob_le_of_pred_imp {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (E F : LocalEvent d) (himp : forall omega, E.pred omega -> F.pred omega) : Prob p E <= Prob p F
prob_union_le [private]	line 27: private theorem prob_union_le {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (E F : LocalEvent d) : Prob p (E.union F) <= Prob p E + Prob p F

Name	Statement or signature
prob_finsetUnionEvent_le_sum [private]	line 67: private theorem prob_finsetUnionEvent_le_sum {d : Nat} {alpha : Type*} [DecidableEq alpha] {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (s : Finset alpha) (E : alpha -> LocalEvent d) : Prob p (finsetUnionEvent s E) <= s.sum fun i => Prob p (E i)
prob_nonneg [private]	line 114: private theorem prob_nonneg {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (E : LocalEvent d) : 0 <= Prob p E
prob_edgeOpenEvent_eq [private]	line 119: private theorem prob_edgeOpenEvent_eq {d : Nat} (p : Real) (b : Bond d) : Prob p (edgeOpenEvent b) = p
mem_of_mem_internalBonds_carrier [private]	line 163: private theorem mem_of_mem_internalBonds_carrier {d : Nat} {S : Finset (Vertex d) } {b : Bond d} {x : Vertex d} (hb : b in internalBonds S) (hx : x in b.carrier) : x in S
clusterIncidentBonds_subset_internalBonds [private]	line 183: private theorem clusterIncidentBonds_subset_internalBonds {d : Nat} {S C : Finset (Vertex d)} : clusterIncidentBonds S C <= internalBonds S
exists_carrier_mem_of_clusterIncidentBonds [private]	line 200: private theorem exists_carrier_mem_of_clusterIncidentBonds {d : Nat} {S C : Finset (Vertex d)} {b : Bond d} (hb : b in clusterIncidentBonds S C) : exists x, x in b.carrier /\ x in C
disjoint_clusterIncident_boundaryBond [private]	line 221: private theorem disjoint_clusterIncident_boundaryBond {d : Nat} {S C : Finset (Vertex d)} {e : OrientedEdge d} (he : e in orientedBoundary S) : Disjoint (clusterIncidentBonds S C) ({bondOfAdj (orientedBoundary_adj he)} : Finset (Bond d))
disjoint_clusterIncident_internal_sdiff [private]	line 241: private theorem disjoint_clusterIncident_internal_sdiff {d : Nat} (T S C : Finset (Vertex d)) : Disjoint (clusterIncidentBonds S C) (internalBonds (T \ C))
disjoint_boundaryBond_internal_sdiff [private]	line 251: private theorem disjoint_boundaryBond_internal_sdiff {d : Nat} {T S C : Finset (Vertex d)} {e : OrientedEdge d} (he : e in orientedBoundary S) (hxC : e.1 in C) : Disjoint ({bondOfAdj (orientedBoundary_adj he)} : Finset (Bond d)) (internalBonds (T \ C))
target_mem_of_connIn [private]	line 270: private theorem target_mem_of_connIn {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {u x : Vertex d} (hconn : ConnIn omega S u x) : x in S
clusterIn_subset [private]	line 276: private theorem clusterIn_subset {d : Nat} (omega : Config d) (S : Finset (Vertex d)) (u : Vertex d) : clusterIn omega S u <= S
connIn_subset [private]	line 283: private theorem connIn_subset {d : Nat} {omega : Config d} {S T : Finset (Vertex d)} {x y : Vertex d} (hST : S <= T) (hconn : ConnIn omega S x y) : ConnIn omega T x y
connToSetIn_subset [private]	line 290: private theorem connToSetIn_subset {d : Nat} {omega : Config d} {S T B : Finset (Vertex d)} {u : Vertex d} (hST : S <= T) (hconn : ConnToSetIn omega S u B) : ConnToSetIn omega T u B
connToSetIn_sdiff_subset [private]	line 297: private theorem connToSetIn_sdiff_subset {d : Nat} {omega : Config d} {T C B : Finset (Vertex d)} {u : Vertex d} (hconn : ConnToSetIn omega (T \ C) u B) : ConnToSetIn omega T u B
connIn_single [private]	line 303: private theorem connIn_single {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {x : Vertex d} (hx : x in S) : ConnIn omega S x x
cluster_subset [private]	line 312: private theorem cluster_subset {d : Nat} {omega : Config d} {S C : Finset (Vertex d)} {u x : Vertex d} (hC : ClusterEqPred S C u omega) (hxC : x in C) : x in S
conn_of_mem_cluster [private]	line 320: private theorem conn_of_mem_cluster {d : Nat} {omega : Config d} {S C : Finset (Vertex d)} {u x : Vertex d} (hC : ClusterEqPred S C u omega) (hxC : x in C) : ConnIn omega S u x
mem_cluster_of_conn [private]	line 329: private theorem mem_cluster_of_conn {d : Nat} {omega : Config d} {S C : Finset (Vertex d)} {u x : Vertex d} (hC : ClusterEqPred S C u omega) (hconn : ConnIn omega S u x) : x in C
sum_clusterEvent_le_connIn [private]	line 338: private theorem sum_clusterEvent_le_connIn {d : Nat} (S : Finset (Vertex d)) (u x : Vertex d) {p : Real} : ((S.powerset.filter fun C => x in C).sum (fun C => Prob p (clusterEvent S C u))) <= Prob p (connInEvent S u x)
connIn_append_open_adj_same [private]	line 432: private theorem connIn_append_open_adj_same {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {x y z : Vertex d} (hzS : z in S) (hyz : Adj y z) (hopen : omega (bondOfAdj hyz) = true) (hconn : ConnIn omega S x y) : ConnIn omega S x z
cluster_closed_under_open_adj [private]	line 469: private theorem cluster_closed_under_open_adj {d : Nat} {omega : Config d} {S C : Finset (Vertex d)} {u x y : Vertex d} (hC : ClusterEqPred S C u omega) (hxC : x in C) (hyS : y in S) (hxy : Adj x y) (hopen : omega (bondOfAdj hxy) = true) : y in C

Name	Statement or signature
path_last_exit_cluster [private]	line 476: private theorem path_last_exit_cluster {d : Nat} {omega : Config d} {C T : Finset (Vertex d)} : forall {gamma : List (Vertex d)} {x b : Vertex d}, PathFromTo gamma x b -> PathIn T gamma -> OpenPath omega gamma -> (exists z, z in gamma /\ z in C) -> b notin C -> exists a y : Vertex d, exists hxy : Adj a y, a in C /\ y notin C /\ omega (bondOfAdj hxy) = true /\ ConnIn omega (T \ C) y b
connToSet_cluster_boundary_witness [private]	line 540: private theorem connToSet_cluster_boundary_witness {d : Nat} {T S B C : Finset (Vertex d)} {u : Vertex d} {omega : Config d} (hu : u in S) (hBS : Disjoint B S) (hconn : ConnToSetIn omega T u B) (hC : ClusterEqPred S C u omega) : exists e, exists he : e in orientedBoundary S, e.1 in C /\ omega (bondOfAdj (orientedBoundary_adj he)) = true /\ ConnToSetIn omega (T \ C) e.2 B
boundary_triple_prob_le [private]	line 575: private theorem boundary_triple_prob_le {d : Nat} (T S B C : Finset (Vertex d)) (u : Vertex d) {e : OrientedEdge d} (he : e in orientedBoundary S) (hxC : e.1 in C) {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) : Prob p (((clusterEvent S C u).inter (edgeOpenEvent (bondOfAdj (orientedBoundary_adj he)))).inter (connToSetInEvent (T \ C) e.2 B)) <= Prob p (clusterEvent S C u) * p * Prob p (connToSetInEvent T e.2 B)
boundary_cluster_value_bound [private]	line 616: private theorem boundary_cluster_value_bound {d : Nat} (T S B C : Finset (Vertex d)) (u : Vertex d) (hu : u in S) (hBS : Disjoint B S) {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) : Prob p ((clusterEvent S C u).inter (connToSetInEvent T u B)) <= (orientedBoundary S).sum (fun e => if _ : e.1 in C then Prob p (clusterEvent S C u) * p * Prob p (connToSetInEvent T e.2 B) else 0)
boundary_inequality_cluster_expansion [private]	line 680: private theorem boundary_inequality_cluster_expansion {d : Nat} (T S B : Finset (Vertex d)) (u : Vertex d) (hu : u in S) (_hST : S <= T) (_hBT : B <= T) (hBS : Disjoint B S) {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) : Prob p (connToSetInEvent T u B) <= (orientedBoundary S).sum (fun e => p * Prob p (connInEvent S u e.1) * Prob p (connToSetInEvent T e.2 B)) -- Cluster last-exit expansion for the finite-volume boundary inequality.
boundary_inequality	line 823: theorem boundary_inequality {d : Nat} (T S B : Finset (Vertex d)) (u : Vertex d) (hu : u in S) (hST : S <= T) (hBT : B <= T) (hBS : Disjoint B S) {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) : Prob p (connToSetInEvent T u B) <= (orientedBoundary S).sum (fun e => p * Prob p (connInEvent S u e.1) * Prob p (connToSetInEvent T e.2 B)) -- Lemma 4, finite-volume boundary expansion inequality.

Sharpness/Subcritical.lean

Imports. Sharpness.BoundaryIneq, Sharpness.FiniteGeometry, Sharpness.Phi.

Definitions, structures, and abbreviations.

Name	Statement or signature
expScale [private, noncomputable]	line 177: private noncomputable def expScale (a : Real) (n : Nat) : Real
translateVertexEmbedding [private]	line 357: private def translateVertexEmbedding {d : Nat} (a : Vertex d) : Vertex d embeds Vertex d
translateBond [private]	line 406: private def translateBond {d : Nat} (a : Vertex d) (b : Bond d) : Bond d
translateInternalBondsEquiv [private]	line 490: private def translateInternalBondsEquiv {d : Nat} (a : Vertex d) (S : Finset (Vertex d)) : internalBonds S ~= internalBonds (S.image (translateVertex a))
boxExitTargets [private, noncomputable]	line 719: private noncomputable def boxExitTargets (d n : Nat) : Finset (Vertex d)
boxExitAmbient [private, noncomputable]	line 722: private noncomputable def boxExitAmbient (d n : Nat) : Finset (Vertex d)

Theorems and lemmas.

Name	Statement or signature
bernWeight_zero_of_all_false [private]	line 13: private theorem bernWeight_zero_of_all_false {alpha : Type*} [Fintype alpha] (sigma : alpha -> Bool) (hsigma : forall e, sigma e = false) : bernWeight (0 : Real) sigma = 1

Name	Statement or signature
bernWeight_zero_of_exists_true [private]	line 20: private theorem bernWeight_zero_of_exists_true {alpha : Type*} [Fintype alpha] (sigma : alpha -> Bool) (hsigma : exists e, sigma e = true) : bernWeight (0 : Real) sigma = 0
exists_true_of_ne_all_false [private]	line 28: private theorem exists_true_of_ne_all_false {alpha : Type*} (sigma : alpha -> Bool) (hne : sigma != fun _ => false) : exists e, sigma e = true
local_prob_zero_of_all_false_not [private]	line 40: private theorem local_prob_zero_of_all_false_not {d : Nat} (E : LocalEvent d) (hfalse : not E.pred (extendConfig E.support (fun _ : E.support => false))) : Prob (0 : Real) E = 0
boxExitProb_zero [private]	line 56: private theorem boxExitProb_zero (d n : Nat) : boxExitProb d (0 : Real) n = 0
boxExitProb_le_one [private]	line 67: private theorem boxExitProb_le_one {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (n : Nat) : boxExitProb d p n <= 1
bernWeight_all_false_pos [private]	line 78: private theorem bernWeight_all_false_pos {alpha : Type*} [Fintype alpha] {p : Real} (hp1 : p < 1) : 0 < bernWeight p (fun _ : alpha => false)
bernProb_pos_of_all_false [private]	line 87: private theorem bernProb_pos_of_all_false {alpha : Type*} [DecidableEq alpha] [Fintype alpha] {p : Real} {A : (alpha -> Bool) -> Prop} (hp0 : 0 <= p) (hp1 : p < 1) (hA : A (fun _ : alpha => false)) : 0 < bernProb p A
local_prob_lt_one_of_all_false_not [private]	line 111: private theorem local_prob_lt_one_of_all_false_not {d : Nat} {p : Real} (E : LocalEvent d) (hp0 : 0 <= p) (hp1 : p < 1) (hfalse : not E.pred (extendConfig E.support (fun _ : E.support => false))) : Prob p E < 1
boxExitProb_lt_one_of_lt_one [private]	line 128: private theorem boxExitProb_lt_one_of_lt_one {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p < 1) (n : Nat) : boxExitProb d p n < 1
div_step_eq [private]	line 142: private theorem div_step_eq {n L : Nat} (hL : 0 < L) (hn : L <= n) : (n - L) / L + 1 = n / L
recurrence_iterate [private]	line 152: private theorem recurrence_iterate {a : Nat -> Real} {rho : Real} {L : Nat} (hL : 0 < L) (hrho0 : 0 <= rho) (hone : forall n : Nat, a n <= 1) (hrec : forall n : Nat, L <= n -> a n <= rho * a (n - L)) : forall n : Nat, a n <= rho ^ (n / L)
exp_bound_single [private]	line 180: private theorem exp_bound_single {a c : Real} {n : Nat} (_ha0 : 0 <= a) (ha1 : a < 1) (_hc0 : 0 <= c) (hc : c <= expScale a n) : a <= Real.exp (-(c * (n : Real)))
exists_small_exp_bound [private]	line 206: private theorem exists_small_exp_bound {a : Nat -> Real} {L : Nat} (hnonneg : forall n : Nat, 0 <= a n) (hlt_one : forall n : Nat, n < L -> a n < 1) : exists c : Real, 0 < c /\ forall n : Nat, n < L -> a n <= Real.exp (-(c * (n : Real)))
rho_pow_le_exp [private]	line 247: private theorem rho_pow_le_exp {rho c : Real} {L n : Nat} (hL : 0 < L) (hrho0 : 0 <= rho) (hrho1 : rho < 1) (hn : L <= n) (hc : c <= if rho = 0 then 1 else -Real.log rho / (2 * (L : Real))) : rho ^ (n / L) <= Real.exp (-(c * (n : Real)))
recurrence_exponential_bound [private]	line 304: private theorem recurrence_exponential_bound {a : Nat -> Real} {rho : Real} {L : Nat} (hL : 0 < L) (hrho0 : 0 <= rho) (hrho1 : rho < 1) (hnonneg : forall n : Nat, 0 <= a n) (hone : forall n : Nat, a n <= 1) (hsmall : forall n : Nat, n < L -> a n < 1) (hrec : forall n : Nat, L <= n -> a n <= rho * a (n - L)) : exists c : Real, 0 < c /\ forall n : Nat, a n <= Real.exp (-(c * (n : Real)))
prob_le_of_pred_imp [private]	line 339: private theorem prob_le_of_pred_imp {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (E F : LocalEvent d) (himp : forall omega, E.pred omega -> F.pred omega) : Prob p E <= Prob p F
translateVertex_neg_apply [private]	line 367: private theorem translateVertex_neg_apply {d : Nat} (a x : Vertex d) : translateVertex (-a) (translateVertex a x) = x
translateVertex_neg_self [private]	line 373: private theorem translateVertex_neg_self {d : Nat} (x : Vertex d) : translateVertex (-x) x = (0 : Vertex d)
translateVertex_neg_eq_sub [private]	line 379: private theorem translateVertex_neg_eq_sub {d : Nat} (x y : Vertex d) : translateVertex (-y) x = x - y
translateVertex_neg_image [private]	line 385: private theorem translateVertex_neg_image {d : Nat} (a : Vertex d) (S : Finset (Vertex d)) : (S.image (translateVertex a)).image (translateVertex (-a)) = S
translateBond_neg_apply [private]	line 423: private theorem translateBond_neg_apply {d : Nat} (a : Vertex d) (b : Bond d) : translateBond (-a) (translateBond a b) = b
translateBond_bondOfAdj [private]	line 446: private theorem translateBond_bondOfAdj {d : Nat} (a : Vertex d) {x y : Vertex d} (hxy : Adj x y) : translateBond a (bondOfAdj hxy) = bondOfAdj ((adj_translate_iff (a
translateBond_mem_internalBonds [private]	line 454: private theorem translateBond_mem_internalBonds {d : Nat} {a : Vertex d} {S : Finset (Vertex d)} {b : Bond d} (hb : b in internalBonds S) : translateBond a b in internalBonds (S.image (translateVertex a))

Name	Statement or signature
translateBond_mem_internalBonds_neg [private]	line 483: private theorem translateBond_mem_internalBonds_neg {d : Nat} {a : Vertex d} {S : Finset (Vertex d)} {b : Bond d} (hb : b in internalBonds (S.image (translateVertex a))) : translateBond (-a) b in internalBonds S
openPath_translate_config [private]	line 504: private theorem openPath_translate_config {d : Nat} {omega omega' : Config d} {a : Vertex d} {gamma : List (Vertex d)} (hopen_translate : forall {u v : Vertex d} (huv : Adj u v), omega (bondOfAdj huv) = true -> omega' (bondOfAdj ((adj_translate_iff (a
connIn_translate_config [private]	line 516: private theorem connIn_translate_config {d : Nat} {omega omega' : Config d} {S : Finset (Vertex d)} {a x y : Vertex d} (hopen_translate : forall {u v : Vertex d} (huv : Adj u v), omega (bondOfAdj huv) = true -> omega' (bondOfAdj ((adj_translate_iff (a
connToSetIn_translate_assignment [private]	line 527: private theorem connToSetIn_translate_assignment {d : Nat} {a : Vertex d} {T B : Finset (Vertex d)} {u : Vertex d} (tau : internalBonds (T.image (translateVertex a)) -> Bool) : let sigma : internalBonds T -> Bool
prob_connToSetIn_le_translate [private]	line 558: private theorem prob_connToSetIn_le_translate {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (a : Vertex d) (T B : Finset (Vertex d)) (u : Vertex d) : Prob p (connToSetInEvent T u B) <= Prob p (connToSetInEvent (T.image (translateVertex a)) (translateVertex a u) (B.image (translateVertex a)))
connIn_single [private]	line 598: private theorem connIn_single {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {x : Vertex d} (hx : x in S) : ConnIn omega S x x
connIn_prepend_open_adj [private]	line 607: private theorem connIn_prepend_open_adj {d : Nat} {omega : Config d} {S : Finset (Vertex d)} {x y z : Vertex d} (hxS : x in S) (hxy : Adj x y) (hopen : omega (bondOfAdj hxy) = true) (hconn : ConnIn omega S y z) : ConnIn omega S x z
connIn_append_open_adj [private]	line 633: private theorem connIn_append_open_adj {d : Nat} {omega : Config d} {S T : Finset (Vertex d)} {x y z : Vertex d} (hST : S <= T) (hzT : z in T) (hyz : Adj y z) (hopen : omega (bondOfAdj hyz) = true) (hconn : ConnIn omega S x y) : ConnIn omega T x z
exitPred_of_connIn_to_not_mem [private]	line 670: private theorem exitPred_of_connIn_to_not_mem {d : Nat} {omega : Config d} {S T : Finset (Vertex d)} {x z : Vertex d} (hxS : x in S) (hzS : z notin S) (hconn : ConnIn omega T x z) : exists e, exists he : e in orientedBoundary S, ConnIn omega S x e.1 /\ omega (bondOfAdj (orientedBoundary_adj he)) = true
mem_boxExitTargets_not_ball [private]	line 725: private theorem mem_boxExitTargets_not_ball {d n : Nat} {x : Vertex d} (hx : x in boxExitTargets d n) : x notin ball d n
boxExitProb_le_boundary_conn [private]	line 732: private theorem boxExitProb_le_boundary_conn {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (n : Nat) : boxExitProb d p n <= Prob p (connToSetInEvent (boxExitAmbient d n) (0 : Vertex d) (boxExitTargets d n))
connToSetIn_le_boxExitProb_of_targets_outside [private]	line 756: private theorem connToSetIn_le_boxExitProb_of_targets_outside {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (T B : Finset (Vertex d)) (k : Nat) (hB : forall z, z in B -> z notin ball d k) : Prob p (connToSetInEvent T (0 : Vertex d) B) <= boxExitProb d p k
connToBoxBoundary_le_shifted_boxExitProb [private]	line 769: private theorem connToBoxBoundary_le_shifted_boxExitProb {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) {L n : Nat} {y : Vertex d} (hy : y in ball d L) (hn : L <= n) : Prob p (connToSetInEvent (boxExitAmbient d n) y (boxExitTargets d n)) <= boxExitProb d p (n - L)
boxExitProb_recurrence_of_phi_lt_one [private]	line 795: private theorem boxExitProb_recurrence_of_phi_lt_one {d : Nat} {p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) (S : Finset (Vertex d)) (L : Nat) (hL : 0 < L) (hS : forall x, x in S -> x in ball d (L - 1)) (h0S : (0 : Vertex d) in S) (_hphi : phi p S < 1) : forall n : Nat, L <= n -> boxExitProb d p n <= phi p S * boxExitProb d p (n - L)
exists_ball_bound_for_finset [private]	line 863: private theorem exists_ball_bound_for_finset {d : Nat} (S : Finset (Vertex d)) : exists L : Nat, 0 < L /\ forall x, x in S -> x in ball d (L - 1)
exponential_decay_below_pTilde_core [private]	line 875: private theorem exponential_decay_below_pTilde_core {d : Nat} {p : Real} (hp0 : 0 < p) (hp : p < pTilde d) : exists c : Real, 0 < c /\ forall n : Nat, boxExitProb d p n <= Real.exp (-(c * (n : Real)))
exponential_decay_below_pTilde	line 905: theorem exponential_decay_below_pTilde {d : Nat} {p : Real} (hp0 : 0 <= p) (hp : p < pTilde d) : exists c : Real, 0 < c /\ forall n : Nat, boxExitProb d p n <= Real.exp (-(c * (n : Real))) -- Lemma 5, exponential decay below the finite-set critical point.

Sharpness/CriticalPoint.lean

Imports. Sharpness.Subcritical, Sharpness.Supercritical.

Definitions, structures, and abbreviations.

Name	Statement or signature
pCrit [noncomputable]	line 7: noncomputable def pCrit (d : Nat) : Real -- The percolation critical point defined from ‘theta‘.
axisVertex [private]	line 59: private def axisVertex {d : Nat} (i : Fin d) (k : Nat) : Vertex d

Theorems and lemmas.

Name	Statement or signature
pCritSet_bddBelow [private]	line 10: private theorem pCritSet_bddBelow (d : Nat) : BddBelow {p : Real | 0 <= p /\ p <= 1 /\ 0 < theta d p}
bernWeight_one_of_all_true [private]	line 16: private theorem bernWeight_one_of_all_true {alpha : Type*} [Fintype alpha] (sigma : alpha -> Bool) (hsigma : forall e, sigma e = true) : bernWeight (1 : Real) sigma = 1
bernWeight_one_of_exists_false [private]	line 23: private theorem bernWeight_one_of_exists_false {alpha : Type*} [Fintype alpha] (sigma : alpha -> Bool) (hsigma : exists e, sigma e = false) : bernWeight (1 : Real) sigma = 0
exists_false_of_ne_all_true [private]	line 31: private theorem exists_false_of_ne_all_true {alpha : Type*} (sigma : alpha -> Bool) (hne : sigma != fun _ => true) : exists e, sigma e = false
local_prob_one_of_all_true [private]	line 43: private theorem local_prob_one_of_all_true {d : Nat} (E : LocalEvent d) (htrue : E.pred (extendConfig E.support (fun _ : E.support => true))) : Prob (1 : Real) E = 1
axisVertex_zero [private]	line 62: private theorem axisVertex_zero {d : Nat} (i : Fin d) : axisVertex i 0 = (0 : Vertex d)
axisVertex_l1 [private]	line 67: private theorem axisVertex_l1 {d : Nat} (i : Fin d) (k : Nat) : l1 (axisVertex i k) = k
axisVertex_adj_succ [private]	line 78: private theorem axisVertex_adj_succ {d : Nat} (i : Fin d) (k : Nat) : Adj (axisVertex i k) (axisVertex i (k + 1))
List_range_succ_head [private]	line 89: private theorem List_range_succ_head (n : Nat) : (List.range (n + 1)).head? = some 0
axisPath_isPath [private]	line 99: private theorem axisPath_isPath {d : Nat} (i : Fin d) (n : Nat) : IsPath ((List.range (n + 1)).map (axisVertex i))
axisPath_head [private]	line 107: private theorem axisPath_head {d : Nat} (i : Fin d) (n : Nat) : ((List.range (n + 1)).map (axisVertex i)).head? = some (0 : Vertex d)
axisPath_last [private]	line 112: private theorem axisPath_last {d : Nat} (i : Fin d) (n : Nat) : ((List.range (n + 1)).map (axisVertex i)).getLast? = some (axisVertex i n)
axisPath_pathFromTo [private]	line 117: private theorem axisPath_pathFromTo {d : Nat} (i : Fin d) (n : Nat) : PathFromTo ((List.range (n + 1)).map (axisVertex i)) (0 : Vertex d) (axisVertex i n)
axisPath_pathIn [private]	line 122: private theorem axisPath_pathIn {d : Nat} (i : Fin d) (n : Nat) : PathIn (ball d n) ((List.range (n + 1)).map (axisVertex i))
axisPath_open_extend_exitSupport [private]	line 130: private theorem axisPath_open_extend_exitSupport {d : Nat} (i : Fin d) (n : Nat) : OpenPath (extendConfig (exitEvent (ball d n) (zero_mem_ball d n)).support (fun _ => true)) ((List.range (n + 1)).map (axisVertex i))
exitPred_all_open_support [private]	line 160: private theorem exitPred_all_open_support {d : Nat} (hd : 0 < d) (n : Nat) : ExitPred (ball d n) (extendConfig (exitEvent (ball d n) (zero_mem_ball d n)).support (fun _ => true))
boxExitProb_one_of_pos_dim [private]	line 189: private theorem boxExitProb_one_of_pos_dim {d : Nat} (hd : 0 < d) (n : Nat) : boxExitProb d (1 : Real) n = 1
pCritSet_nonempty_of_pos_dim [private]	line 195: private theorem pCritSet_nonempty_of_pos_dim {d : Nat} (_hd : 0 < d) : ({p : Real | 0 <= p /\ p <= 1 /\ 0 < theta d p} : Set Real).Nonempty
pCrit_le_one [private]	line 204: private theorem pCrit_le_one (d : Nat) : pCrit d <= 1
pCrit_le_of_mem [private]	line 219: private theorem pCrit_le_of_mem {d : Nat} {p : Real} (hp : 0 <= p /\ p <= 1 /\ 0 < theta d p) : pCrit d <= p
pCrit_le_pTilde [private]	line 224: private theorem pCrit_le_pTilde (d : Nat) : pCrit d <= pTilde d
pTilde_le_pCrit [private]	line 252: private theorem pTilde_le_pCrit {d : Nat} (hd : 0 < d) : pTilde d <= pCrit d
pTilde_eq_pCrit	line 267: theorem pTilde_eq_pCrit {d : Nat} (hd : 0 < d) : pTilde d = pCrit d -- The finite-set critical point equals the ‘theta‘ critical point in nontrivial dimension.

Name	Statement or signature
exponential_decay_below_pCrit	line 271: theorem exponential_decay_below_pCrit {d : Nat} {p : Real} (hd : 0 < d) (hp0 : 0 <= p) (hp : p < pCrit d) : exists c : Real, 0 < c /\ forall n : Nat, boxExitProb d p n <= Real.exp (-(c * (n : Real))) -- Subcritical exponential decay rewritten with ‘pCrit‘.
supercritical_lower_bound_above_pCrit	line 280: theorem supercritical_lower_bound_above_pCrit {d : Nat} {p : Real} (hd : 0 < d) (hp : pCrit d < p) (hp1 : p < 1) : theta d p >= (p - pCrit d) / (p * (1 - pCrit d)) -- Supercritical density lower bound rewritten with ‘pCrit‘.

Sharpness/Main.lean

Imports. Sharpness.CriticalPoint.

Definitions, structures, and abbreviations. *None.*

Theorems and lemmas.

Name	Statement or signature
sharpness_zd	line 9: theorem sharpness_zd (d : Nat) (_hd : 2 <= d) : (forall p : Real, 0 <= p -> p < pCrit d -> exists c : Real, 0 < c /\ forall n : Nat, boxExitProb d p n <= Real.exp (-(c * (n : Real)))) /\ (forall p : Real, pCrit d < p -> p < 1 -> theta d p >= (p - pCrit d) / (p * (1 - pCrit d))) -- Sharpness for nearest-neighbor Bernoulli bond percolation on ‘Z^d‘, stated using finite box exit probabilities and ‘theta‘ as their decreasing-limit infimum.